Slim Domain

A Domain-Structuring Pattern for .NET That Leaves Inadvertent Technical Debt Nowhere to Accrue

Persistence contracts separated from business behavior — and the default paths to technical debt closed by construction

Paul Smith

Funcular Labs

Version 1.0 — Release Candidate 4 — October 2026

Prepared with AI writing and review assistance; see Appendix A for the methodology and disclosure statement.

Table of Contents

Abstract 5
1. Introduction 6
1.1 The Recurring Pain Point 6
1.2 Why This Pattern Exists 7
1.3 Provenance and Evidence 8
1.4 Methodology, Scope, and Disclosure 8
1.5 Reading Guide 9
2. What Slim Domain Is — and What It Is Not 10
2.1 What It Is 10
2.2 What It Is Not 10
2.3 Boundaries, Seams, and the Legitimate DTO 11
2.4 Where It Fits: Compatible Architectures 14
2.5 Where Friction Arises 16
2.6 Adoption Paths: The Habit, Not the Library 17
2.7 Why Try It 18
3. The Problems This Pattern Was Designed to Solve 19
3.1 Onboarding Speed and Reduced Learning Curve 19
3.2 Reducing Bug Injection Across Experience Levels 20
3.3 High-Security and Regulated Environments 20
3.4 Technical Debt as the Unifying Lens 21
4. Core Principles of Slim Domain 22
4.1 Slim Entities as Pure Data Contracts 23
4.2 Enriched Business Objects via Inheritance or Composition 24
4.3 Service-Gated Persistence 27
4.4 Use-Case-Shaped Reads, and the Attribute Road's Write Convenience 29
4.5 How the Principles Map to SOLID 31
4.6 The Generator Contract 32
5. Micro-ORMs vs. Monolithic ORMs — A Strong Recommendation 33
5.1 Why Micro-ORMs Align with Slim Domain's Goals 33
5.2 The Candidates 34
5.3 Feature Comparison Matrix 34
6. Advanced Techniques in Practice 36

6.1 Loading Object Graphs Without Navigation Properties 36
6.2 Transactions: Implicit, Atomic, Named, and Multi-Call 38
6.3 Non-Canonical Entities: Detail Classes, Views, and Multi-Table Shapes 41
6.4 Intersection-Mapping Exceptions and Data Annotations 43
6.5 Inheritance Coupling: A Deeper Look 44
6.6 Testing the Layers 45
6.7 Optimistic Concurrency with Intersection Mapping 47
6.8 Schema-Derived Interfaces: Centralizing Cross-Cutting Entity Concerns 48
6.9 The Regulated Seam in Practice 52
7. Consequences and Required Discipline 55
7.1 What the Pattern Provides 55
7.2 What It May Enable 56
7.3 Costs and the Discipline They Require 56
8. Comparative Analysis: Slim Domain vs. Strict N-Tier 58
8.1 Summary Comparison 58
8.2 Assessment 59
8.3 Against Modern Baselines 60
9. Presenting Slim Domain in N-Tier Terms 61
9.1 Current Aegis Structure (Slim Domain Naming) 62
9.2 Step 1: Rename-Only Restructuring (High Perception Gain, Zero Code Change) 62
9.3 Step 2: Optional Thin Repository Layer (Full Classic Appearance) 63
9.4 Enforcement: Making the Policy Verifiable 64
10. Related Work 66
11. Limitations and Validation Roadmap 67
11.1 Limitations 67
11.2 A Validation Roadmap 69
12. Conclusion 70
References 71
Appendix A: Review Methodology and AI Assistance Disclosure 73
A.1 Adversarial Review Process 73
A.2 Authorship and AI Assistance 74
Appendix B: Structural Metrics from Paired Production Systems 74
B.1 Systems and Method 74

B.2 The Architectural Signal 75
B.3 Persistence Dispersion 76
B.4 Complexity: A Control, Not a Scoreboard 77
B.5 Test Corpora 78
B.6 Threats to Validity 78

Abstract

Slim Domain is a domain-structuring pattern for .NET systems — not an application architecture and not a library. Relative to a disciplined service layer over Dapper or Entity Framework Core, it changes five things: reads are shaped to the use case rather than to the object graph; the canonical entity is the table and nothing more; business objects acquire the entity shape by inheriting or wrapping it inside a trust boundary, so in-process DTO mapping disappears except where a defined boundary warrants it; services alone persist, and nouns never save themselves; and recurring column groups become generator-proposed, human-approved interfaces that centralize cross-cutting rules such as audit stamping. Read through the lens of technical debt, the five changes have one effect: the common forms of inadvertent debt — over-fetching, parallel shapes, ambient persistence, scattered cross-cutting rules — become structurally unavailable rather than merely discouraged, and the interest paid on cross-cutting change is bounded by construction. It is intended for teams of mixed experience levels, including those common in regulated, high-security environments.

The pattern rests on four principles: *slim entities* as pure data contracts; *enriched business objects*, via inheritance or composition, that acquire the persistence shape and add behavior; *service-gated persistence*, in which the services layer is the sole consumer of the data-access library; and *use-case-shaped reads*, in which every query returns a shape carrying only what the operation uses. Where a library maps only the intersection of columns and properties, business objects need no per-member mapping attributes even on writes — a convenience the pattern welcomes and does not require.

This paper defines the pattern and illustrates it with C# examples drawn from a fictitious reference solution, the Aegis Platform. It states what the pattern is and is not, which application architectures it fits inside and where it meets friction, and how teams on any data-access library can adopt its central habit; develops technical debt as the lens that unifies its tenets (Section 3.4); provides a measured feature comparison of leading micro-ORMs; demonstrates practical techniques for object-graph loading, transactions, non-canonical projections, and mapping exceptions; states the pattern's consequences and the discipline each cost requires, with particular attention to healthcare and financial services; compares the discipline the pattern requires against classic strict n-tier layering; and shows how teams can present a Slim Domain solution in conventional n-tier terms — including optional build-time enforcement — without conversion or rework.

The argument rests on mechanism — how each rule constrains the defect classes a regulated team fears — and on a structural measurement of two production systems (Appendix B) that describes what the pattern's rules oblige developers to write, compared with a legacy Entity Framework and active-record codebase from the same lineage. That measurement is not a comparison with a modern service layer and is not offered as proof of superiority; Section 11 states the evidentiary limits and proposes the measurements that should follow. The central claim is narrower: once the deliberate engineering discipline required by any layered architecture is accounted for, Slim Domain changes the default path so that the discipline is easier to apply, easier to audit, and harder to violate accidentally.

---

1. Introduction

1.1 The Recurring Pain Point

Modern .NET development is awash in architectural patterns — Clean Architecture, Onion Architecture, Vertical Slice Architecture, rich Domain-Driven Design, and various flavors of "proper" n-tier. Each is promoted with strong claims of superiority, yet many development teams continue to struggle with the same classes of defects: N+1 queries, unintended state mutations, complex event-sequencing bugs, inconsistent implementations that accumulate technical debt, and slow onboarding for new or junior developers.

Architects and principal engineers rarely control team membership. We inherit developers of all experience levels, tenures, and degrees of attention to detail. In high-security environments — healthcare under HIPAA [10], financial services under PCI-DSS [11] and SOX, global banking under data-residency and consent regulations — the cost of a single missed invariant or accidental data exposure is measured not only in dollars but in regulatory exposure and human trust.

Slim Domain was created to address a specific, practical question: how do we give teams strong guardrails against the most common sources of technical debt and security incidents while keeping the learning curve and institutional ceremony to their bare minimum — and no further?

1.2 Why This Pattern Exists

Most of what a typical line-of-business application loads is never used by the operation that loaded it. A screen that displays six fields materializes an aggregate of sixty; a service that updates one column tracks the state of forty; a report that needs three joined values pulls three object graphs and discards them after reading. The waste is not only bytes on the wire. It is the mapping layers that shuttle those values between shapes, the change trackers that watch them, the navigation properties that invite the next developer to load still more, and the review effort spent policing all of it.

Slim Domain exists to cut data access down to what is actually used or displayed, and to make that reduction the path of least resistance rather than an act of discipline. Everything else in the pattern follows from that purpose and can be stated as five tenets:

- Flatten to the use case. Eschew large — ideally any — object graphs; shape each read to the immediate need.
- Keep canonical entities minimal. The persistence contract is the table and nothing more; richer shapes live elsewhere.
- Inherit where boundaries do not require separation. Business objects acquire the entity shape directly, minimizing DTOs and value transference; DTOs are reserved for genuine boundaries.
- Keep nouns and verbs apart. Objects carry state and behavior; actors — services — perform persistence. Active-record styles, in which the noun saves itself, are avoided.
- Structure the domain; do not dictate the architecture. The pattern coexists with vertical-slice, onion/concentric, layered-monolith, and microservice architectures rather than competing with them.

Stated in the vocabulary of technical debt [25], [26]: every over-fetched graph, every parallel shape, and every ambient save is principal borrowed against future change, and the interest is paid on every change thereafter. Slim Domain exists to make those borrowings structurally unavailable rather than merely discouraged. Section 3.4 develops that lens, Appendix B measures its structural proxies, and Section 11.2 proposes the longitudinal measurement that would test it.

Section 2 expands each tenet and draws the pattern's boundaries; Sections 3 and 4 supply the problem background and the formal principles.

1.3 Provenance and Evidence

The pattern originates in roughly two decades of production use across line-of-business and high-security regulated systems, applied by developers ranging from recent graduates to principal engineers. That provenance explains the pattern's design pressures — mixed-skill teams, mandatory auditability, fine-grained consent enforcement — but this paper deliberately does not rest its argument on practitioner recollection.

Instead, the evidence offered is primarily structural. Appendix B presents a quantitative comparison of two production systems — a mature classic-stack system (Entity Framework, rich domain model, legacy active-record persistence) and a recent Slim Domain system — measuring complexity, data-access surface, mapping ceremony, and persistence dispersion. The results describe what the pattern's rules oblige developers to write: schema-mapping attribute ceremony roughly an order of magnitude lower per line; change-tracking operations, eager-load sites, and active-record persistence methods absent by construction; persistence participation concentrated from hundreds of business-layer files into seven; and, as a robustness check, cyclomatic complexity at parity at the business-and-data layer. Two limits apply and are repeated wherever the numbers appear: the baseline is a legacy designer-era Entity Framework system with active-record persistence, not a modern service-gated EF Core or Dapper codebase; and the absent constructs are absent in any codebase that leaves Entity Framework. The measurement therefore supports the pattern's tenets within one lineage and does not establish superiority over its real alternatives, which Section 8.3 compares on mechanism. These are measures of structural surface, not of defect outcomes; Section 11.2 proposes the outcome measurements that should follow.

1.4 Methodology, Scope, and Disclosure

Drafts of this paper were subjected to iterative adversarial review using large language models from two vendors — xAI Grok (4.2 and 4.3 beta on early drafts; 4.7 for the current rounds) and Anthropic Claude (Fable 5.1 and Opus 5.5) — prompted to critique the pattern from the perspectives of a software architect, a compliance auditor, a junior developer, and a regulator. That process surfaced several of the objections cataloged in Sections 7 and 8 and sharpened the comparative analysis. Such review complements, but

does not substitute for, human peer review and empirical validation. Appendix A describes the process and the resulting authorship policy in detail.

Disclosure: FunkyORM, one of the micro-ORMs discussed in Section 5, is an open-source (MIT-licensed) library maintained by Funcular Labs, the author's organization [12]. It is cited as one of several viable options; the pattern does not depend on it. Feature descriptions for FunkyORM reflect release 3.12.1 (October 2026) and the project's published documentation.

1.5 Reading Guide

A recurring theme runs throughout this paper: Slim Domain does not eliminate the need for deliberate discipline — it makes that discipline easier to apply, easier to audit, and harder to violate accidentally.

Section 2 states what the pattern is and is not: its tenets; the boundaries at which a transfer object is warranted (Figure 1); the application architectures it fits inside (Figures 2 and 3); where friction arises; and the adoption paths open to teams on any data-access library (Figure 4). Section 3 motivates the pattern and, in Section 3.4, develops technical debt as the lens that unifies its tenets; Section 4 defines the principles. Section 5 examines the micro-ORM landscape, including a feature comparison matrix. Section 6 demonstrates the practical techniques most often raised as objections: loading object graphs without navigation properties, transaction management, non-canonical projections, mapping exceptions, inheritance coupling, testing, optimistic concurrency, schema-derived interfaces for cross-cutting concerns, and the regulated seam in practice. Section 7 states the pattern's consequences and the discipline each cost requires; many of the costs it catalogs exist in classic n-tier as well, and the difference frequently lies in where and how the mitigation must be applied. Section 8 compares the two approaches directly and then against modern EF Core and Dapper baselines. Section 9 shows how to present a Slim Domain solution in conventional n-tier terms — a matter of naming and structure rather than conversion — including optional build-time enforcement. Appendix B presents the structural metrics referenced throughout.

2. What Slim Domain Is — and What It Is Not

Patterns are misapplied most often at their edges, so this section draws the edges explicitly: the tenets in positive form, the things the pattern is routinely mistaken for, the boundaries at which a transfer object is warranted, the application architectures it lives inside, the styles with which it has friction, the routes by which a team on any data-access library can adopt it, and the reasons to try.

2.1 What It Is

- A domain-structuring pattern. It prescribes three roles — minimal canonical entities, behavioral business objects, and persistence-gating services — and the dependency direction among them (Figure 5, Section 4). It says nothing about how requests arrive, how features are organized, or how deployment units are drawn.
- A flattening discipline. Each read is shaped to its use case — a detail entity, a view, or a projection carrying exactly the fields the operation displays or decides on. Object graphs are composed explicitly when genuinely needed (Section 6.1), never loaded ambiently.
- A minimalism rule for canonical entities. The generated entity is the table: its columns and nothing more. Richer read-side shapes are separate, non-canonical classes (Section 6.3), so the canonical contract never grows to serve a screen.
- An inheritance-first stance within a boundary. Where no trust, serialization, or ownership boundary separates producer from consumer, the business object simply is the entity plus behavior. DTOs and mappers are reserved for real boundaries — APIs, messages, other bounded contexts — rather than imposed between layers that share a process.
- A separation of nouns from verbs. Business objects hold state and domain behavior; services hold the verbs of persistence. An object never saves itself.

2.2 What It Is Not

- Not an ORM or a library. FunkyORM is one on-ramp whose attributes make the flattening habit the easy path (Section 2.6); the pattern is equally practicable with Dapper, LinqToDB, or a deliberately constrained Entity Framework.

- Not an application architecture. It does not compete with vertical-slice, onion/clean, layered, or microservice designs; it occupies the domain and persistence seams inside whichever of them a team has chosen (Section 2.4).
- Not a prohibition on relationships. Related data is loaded — explicitly, shaped to the use case. What is rejected is the navigation property as the organizing principle of retrieval (Section 6.1).
- Not a ban on DTOs. Contracts belong at boundaries (Aegis.Shared.Contracts in the reference solution). The pattern removes the DTOs between layers that share a process, not the ones that cross a trust or ownership line; Section 2.3 defines those lines and the factors that justify a transfer.
- Not an anemic domain model. Behavior lives on business objects; what is kept thin is the persistence contract, not the domain (Section 10).
- Not a repository mandate — or a repository ban. Thin, stateless repositories are optional presentation (Section 9.3); fat, stateful ones are the thing to avoid.
- Not a replacement for strategic domain-driven design. Ubiquitous language and bounded contexts apply unchanged; only the tactical insistence on aggregate-centric loading is relaxed.

2.3 Boundaries, Seams, and the Legitimate DTO

The pattern's position on data-transfer objects is easy to misread, so it is worth stating precisely. Slim Domain discourages mapping operations — the copying of values from one shape into another, nearly identical shape — and asks that each one be evaluated with a jaundiced eye. It does not forbid them. The question is never "are DTOs allowed?" but "what makes this particular transfer necessary?" This subsection answers that question for readers who have not spent years arguing about it.

Four terms carry the argument. A *data-transfer object* (DTO) is a behaviorless carrier of values whose shape is chosen for the consumer that receives it, not for the domain that produced it. A *boundary* is a line in a system across which something material changes: who is trusted, who owns the data, how it is serialized, or how long the shapes on each side are expected to live — an HTTP API, a message queue, a file export, another team's bounded context. A *seam* is a place in the code where one behavior can be substituted for another without rewriting the code around it; in Slim Domain the service gate (Section 4.3) is the primary seam, and it is also where boundary decisions are

enforced. A *first-order business object* is the enriched object of Section 4.2 — the entity shape plus behavior and, often, members the enterprise has no wish to publish.

Inside a boundary — between layers of one process that share a trust level, an owner, and a release cycle — the pattern's default is to use the business object itself. Nothing is gained by copying forty properties into a second class so that a controller can read six of them; what is gained is a second shape to keep synchronized and a mapper to review. This is the inheritance-first stance of Section 2.1.

A transfer onto a DTO is warranted when the consumer sits outside a boundary and the business object would otherwise cross that boundary unfiltered. Any one of the following factors is sufficient; several usually apply at once:

- Confidentiality. The business object carries members the enterprise does not want shared — internal identifiers, cost bases, risk tiers, audit stamps, soft-delete markers, regulated personal or health fields beyond the consumer's need. A DTO is an allow-list: only what is named leaves. Serializing the business object is a deny-list that must be maintained forever.
- Behavior containment. Business objects carry verbs — ApplyDebit, Suspend — and the invariants that protect them. Outside the API boundary those verbs have no meaning, cannot be invoked safely, and should not be visible at all; a consumer that sees them will reason about them. A DTO has no verbs, so it cannot be mistaken for an actor.
- Contract stability. A published shape must stay stable for consumers the team does not control while the schema and the business object evolve underneath it. The DTO is the versioned contract; the business object stays free to change.
- Wire-format fit. JSON naming, nullability, enumeration representation, date formats, cyclic references, and payload size are constraints of the transport, and the domain shape should not bend to them.
- Ownership and anti-corruption. When data crosses into another bounded context, team, or organization, the receiver must not come to depend on the sender's internal shape — or the sender can never change it. The DTO is the agreed vocabulary at the border.
- Inbound intent and over-posting. Requests arriving from outside express intent — "debit this account by this amount" — and should be bound to a request DTO that says exactly that. Binding client input directly to a business

object invites over-posting: a caller who supplies a Balance or an IsApproved field that the model binder obligingly sets. The request DTO is the pattern's input filter; the service then invokes behavior.

Equally worth naming are the reasons that do not justify a DTO: a layering convention that each layer "should" have its own model; discomfort with coupling between two layers that share a process, owner, and release; a mapping library already installed; and a future API that may one day need a contract. These produce the mapper-everywhere cultures that Figure 3 lists under friction.

One further distinction keeps the discouragement coherent. Mapping a loaded business object into a DTO is the operation the pattern questions. Projecting a query directly into a use-case shape — a detail entity, a view, a narrow Select — is a different operation: no business object is materialized, nothing is copied, and the shape is the pattern's first tenet in action (Section 2.6). Read-side contracts should take that road wherever no behavior is involved, so that the DTO and the use-case shape are the same class. Figure 1 condenses the decision.

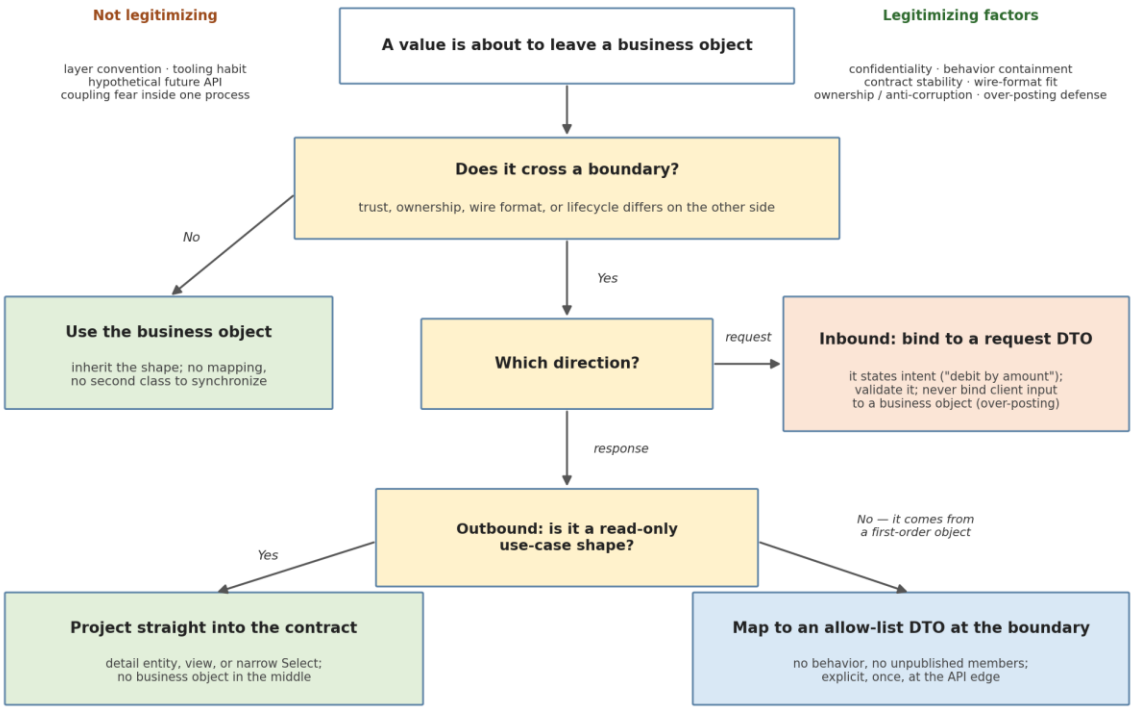


Figure 1. When a DTO is warranted. Inside the boundary, use the business object; across it, publish an allow-list contract — projected directly wherever no behavior is involved.

Listing 1 shows a legitimate transfer in the reference solution. The business object stays behind the boundary with its behavior and unpublished members; the contract names exactly what leaves; inbound intent arrives as a request DTO; and the mapping is written once, explicitly, at the edge — not between internal layers.

```
// First-order business object: carries behavior and members the
// enterprise does not publish (cost basis, risk tier, audit stamps,
// soft-delete markers ...).
public sealed class Account
{
    private readonly AccountEntity _data;      // private state (Listing 4)
    public void ApplyDebit(decimal amount, DateTime nowUtc) { /* rules */ }
    public string AccountNumber => _data.AccountNumber;   // published
    public decimal Balance => _data.Balance;
    public string Status => _data.Status;
    // ... _data also holds CostBasis, RiskTier, DeletedUtc ...
}

// Contract (Aegis.Shared.Contracts): an allow-list, no behavior.
public sealed record AccountResponse(
    string AccountNumber, decimal Balance, string Status);

// Inbound intent arrives as a request DTO, never as the object.
public sealed record DebitRequest(decimal Amount);

// The boundary (an API endpoint) is where the transfer happens:
// explicitly, once, at the edge, from the state the service returns.
[HttpPost("{id}/debit")]
public async Task<AccountResponse> Debit(int id, DebitRequest request)
{
    var account = await _accounts.DebitAsync(
        id, request.Amount, _correlation.Current);

    return new AccountResponse(
        account.AccountNumber, account.Balance, account.Status);
}
```

Listing 1. A legitimate transfer: behavior and unpublished members stay behind the boundary; the contract is an allow-list; intent arrives as a request DTO.

The rule of thumb for a reviewer: when a mapping appears in a pull request, ask which boundary it serves. If the answer names a trust, ownership, format, or lifecycle line, the mapping is doing its job. If the answer is a layer diagram, the pattern suggests deleting it.

2.4 Where It Fits: Compatible Architectures

Figure 2 places the pattern's three roles inside four common application architectures. In each, the persistence gate is whatever construct already mediates between request handling and the domain; the pattern changes what that construct is allowed to do, not where it sits.

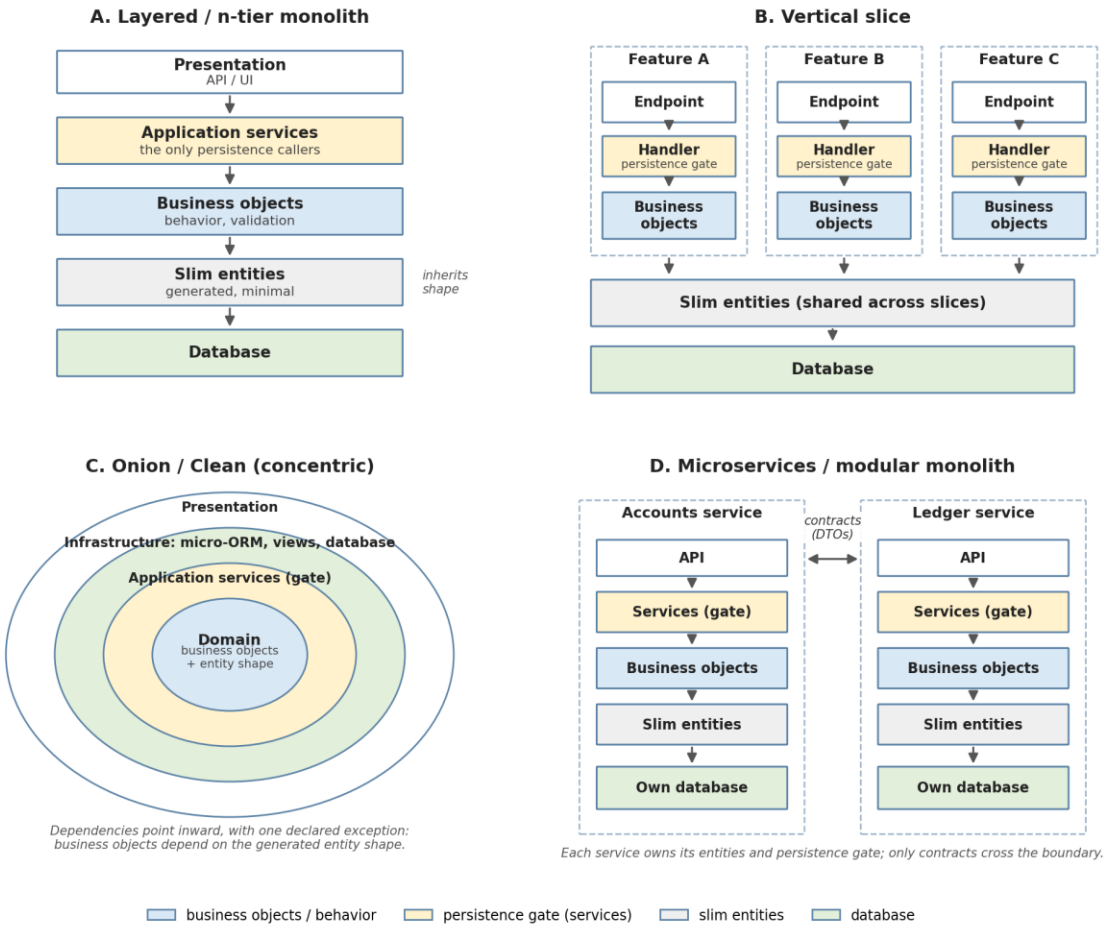


Figure 2. Slim Domain embedded in four application architectures. The three roles keep their relationships; only their host changes.

- Layered / n-tier monolith. The closest fit: application services are already the gate. Section 9 shows that the remaining differences are naming and emphasis.
- Vertical slice. Each feature handler is a gate; business objects are sliced with their features; slim entities are the one deliberately shared layer, because a table is a fact rather than a feature. The combination is natural — slices want narrow, use-case-shaped reads, which is the pattern's first tenet.
- Onion / Clean. Dependencies point inward unchanged, with one declared exception: business objects depend on the generated entity shape (examined in Sections 6.5 and 10). Teams that want the inversion restored can wrap entities in the composition variant (Section 4.2).
- Microservices / modular monolith. Minimal entities are cheap to own per service; each service has its own gate and database; only contracts cross boundaries — exactly where the pattern says DTOs belong.

Command-query separation and event sourcing fit the same way: the service seam is where commands commit, events are emitted, and read models — themselves detail entities — are populated (Section 10).

2.5 Where Friction Arises

Friction appears wherever a style's central assumption is the thing the pattern declines to do: load and navigate an object graph. Figure 3 lists the common cases beside the architectures that fit naturally.

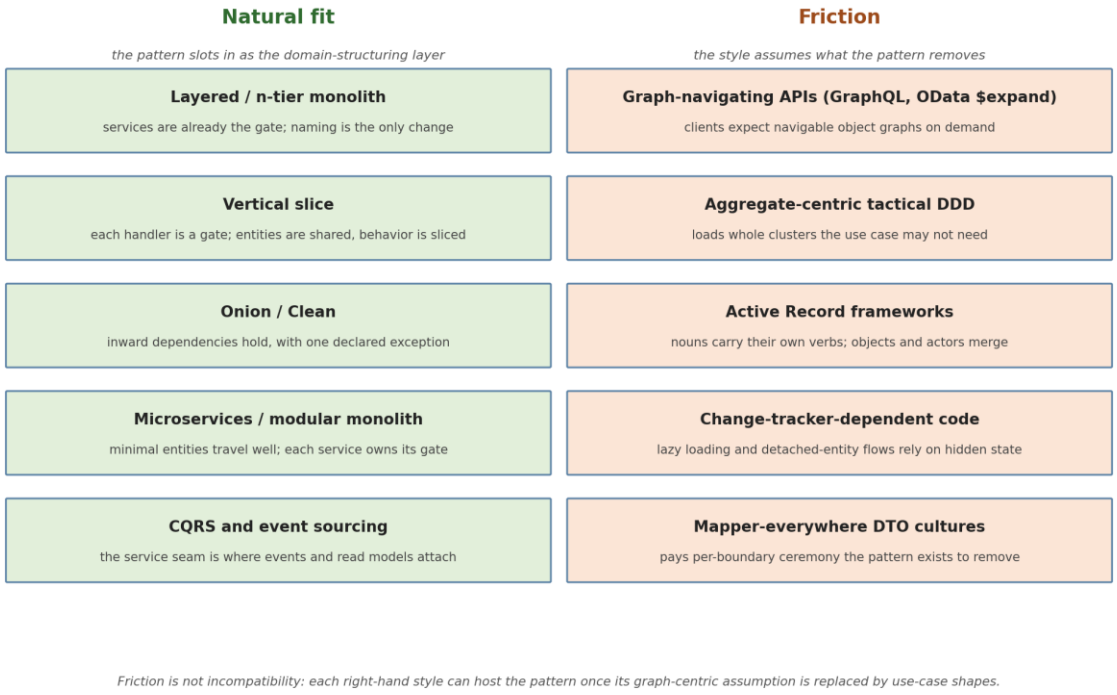


Figure 3. Natural fit versus friction. Friction marks styles whose organizing assumption is the graph the pattern declines to load.

Graph-navigating API styles (GraphQL resolvers, OData $expand) promise clients arbitrary traversal on demand, which an ambient graph serves effortlessly and a flattened read serves only per query shape. Aggregate-centric tactical domain-driven design loads whole consistency clusters where the use case may need one field. Active Record frameworks fuse noun and verb by design. Change-tracker-dependent code — lazy loading, detached-entity round trips — relies on exactly the hidden state the pattern removes. Mapper-everywhere cultures pay per-boundary ceremony between layers that share a process, which the pattern treats as waste.

Friction is not incompatibility. Each of these styles can host the pattern once its graph-centric assumption is replaced by use-case shapes: a resolver can compose from narrow reads; an aggregate can be loaded explicitly where invariants genuinely span it; an active-record codebase can move verbs into services one aggregate at a time. The honest cost is that the host style stops doing the thing it was designed to make easy — and that is a legitimate reason for a team to decide the pattern is not for that codebase.

2.6 Adoption Paths: The Habit, Not the Library

The pattern's core is a habit — cut data access down to what is actually useful or displayed — and a library can at most make that habit the path of least resistance. Figure 4 shows the same flattened use-case shape reached by three roads.

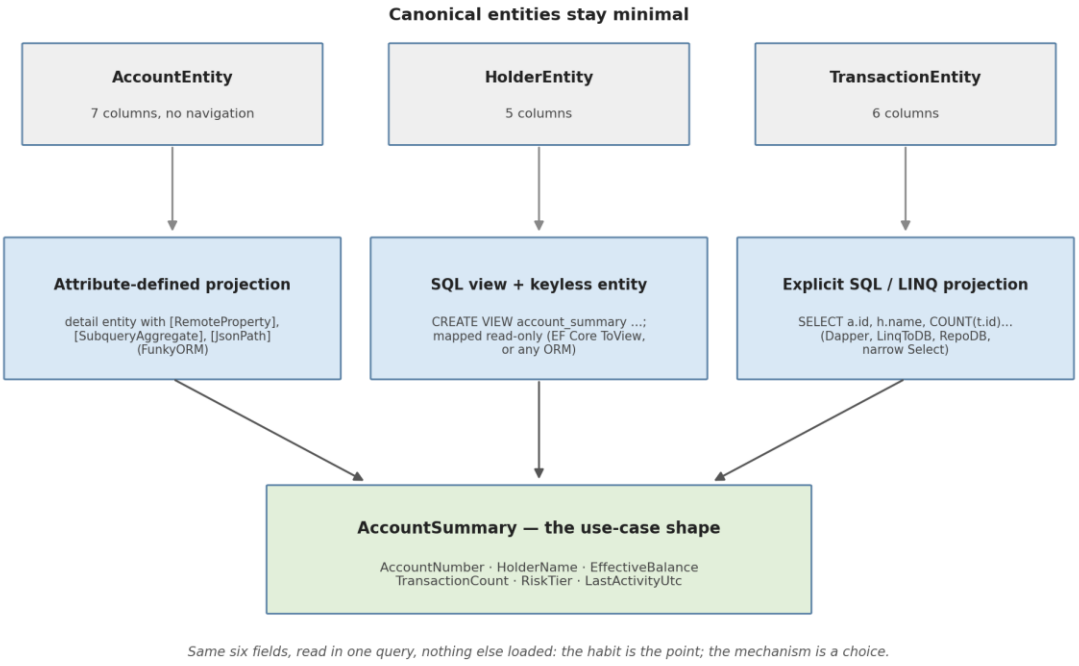


Figure 4. Three mechanisms, one use-case shape. The canonical entities stay minimal regardless of which road a team takes.

The attribute road. FunkyORM's detail entities express the shape in code — [RemoteProperty] for flattened values, [SubqueryAggregate] for child counts, [JsonPath] and [JsonCollection] for semi-structured data, and narrow projections for key lists — with no view to deploy and the result filterable like any entity (Section 6.3). It is the lowest-friction on-ramp, which is why this paper's examples use it; it is one road, not the pattern.

The view road. Entity Framework teams can practice the same habit without changing libraries: map read-side shapes to SQL views as keyless entity types; project with Select into use-case shapes and read them without tracking; disable lazy loading and strip navigation properties from canonical entities, or isolate them to detail types; and confine SaveChanges to services. A monolithic domain — or one subdomain of it — converts to the flatter pattern one view at a time, which is a realistic migration for a large codebase. The change tracker remains, so some of the structural guarantees measured in Appendix B are not available by construction and must be held by convention and by the fitness functions of Section 9.4; the data-access habit, however, transfers intact; Listing 7 shows the write.

The projection road. Dapper, LinqToDB, RepoDB, and their peers express each use-case shape as explicit SQL or a LINQ projection. Intersection-mapping convenience is partial (Table 2), so a write names its columns explicitly — Listing 5 shows the complete debit on Dapper — but the reading habit is native.

Whichever road a team takes, the measure of adoption is not the library in the project file. It is the ratio of fields loaded to fields used.

2.7 Why Try It

The case for a trial is short. The pattern removes whole defect classes by construction rather than by vigilance — against the legacy baseline of Appendix B, the eager-load, change-tracking, and ambient-persistence constructs measure zero, as they would in any codebase that left Entity Framework; what the pattern adds is that nothing takes their place. Against that same baseline it cuts mapping ceremony by roughly an order of magnitude per line of comparable code; against a disciplined modern service layer the delta is smaller and is stated in Section 8.3. It concentrates persistence into a surface an auditor can read in an afternoon. Its reading surface is four concepts (Section 3.1). And it is reversible: because the pattern lives inside an existing architecture rather than replacing it, a trial can be bounded — one subdomain, one feature slice, one new service — presented in whatever naming the organization prefers (Section 9), and measured against the roadmap of Section 11.2 before anything else changes.

The cost side is equally short and is stated in full in Section 11: schema coupling made visible, persistence primitives added deliberately, encapsulation by convention in the inheritance variant, and an evidentiary base that is structural rather than outcome-based. A team that reads both lists and still recognizes its own pain in Section 3 has a

cheap experiment available. In debt terms the experiment tests one proposition: that the interest on cross-cutting change — the files touched, the shapes synchronized, the graphs re-fetched — stays bounded as the codebase grows (Section 3.4).

---

3. The Problems This Pattern Was Designed to Solve

3.1 Onboarding Speed and Reduced Learning Curve

Classic Entity-Framework-centric n-tier and rich domain-driven implementations often require developers to internalize:

- DbContext lifetime management and disposal rules
- Change tracking and identity-map behavior
- Lazy-loading versus eager-loading decision trees
- Aggregate boundaries and repository contracts
- Mapping strategies between entities, DTOs, and view models

Slim Domain deliberately minimizes this surface area. Four concepts suffice to read a Slim Domain codebase:

1. Slim entities define the persisted shape (generated, no behavior).
2. Business objects acquire that shape — by inheriting it or by wrapping it — and add behavior.
3. Services are the only place that ever touches the database.
4. Reads are shaped to the use case, and writes name what they change; where the library maps the intersection of columns and properties, the business object can be persisted directly.

Writing a Slim Domain codebase well takes more than four concepts — the variant choice, the DTO warrant test, detail entities, transaction ownership, the version predicate, and the generator workflow — and Section 6 is that curriculum. The claim is that the reading surface is small enough for the curriculum to be learned on the job, not that the job is four concepts long.

This is not a simplification of architecture for its own sake. It is the removal of *accidental complexity* so that the essential complexity of the business domain can receive full attention, in Brooks's sense of those terms [1].

3.2 Reducing Bug Injection Across Experience Levels

Two of the most common sources of production incidents the author has observed in .NET codebases are:

- N+1 queries and unintended lazy loading, especially with Entity Framework navigation properties
- State-related bugs from change tracking, detached entities, or long-lived DbContext instances

By removing navigation properties from the persistence model entirely and using stateless micro-ORMs, Slim Domain makes these classes of bugs structurally difficult to introduce. The pattern does not rely on developer discipline to avoid them — it makes the dangerous path the inconvenient one.

3.3 High-Security and Regulated Environments

In healthcare (HIPAA), financial services (PCI-DSS, SOX), and global banking (data residency, consent, auditability), the consequences of a missed invariant or accidental data exposure are severe. The pattern's explicit service layer creates natural, auditable seams for:

- Consent checks before protected health information or financial data is materialized
- Encryption and tokenization at the service boundary
- Structured logging and correlation identifiers for every persistence operation
- Insertion of domain events for immutable audit trails

These capabilities are not free — they still require deliberate implementation. The explicit nature of Slim Domain, however, makes consistent enforcement substantially easier than in patterns that rely on hidden ORM behavior or on rich entities that carry decrypted data in memory for extended periods.

A core philosophical position of Slim Domain is that it shifts the burden from *human discipline* to *architectural enforcement*. Requiring every developer to be consistently conscientious, diligent, and insightful at every moment is a far taller order than designing a system in which key rules — business logic, validation, auditing, consent, encryption handling — are enforced at compile time or through clear structural boundaries. We do not always control team membership or the daily state of every

developer's attention. Slim Domain reduces reliance on perfect human performance by making the right thing the easy — or the only — thing.

3.4 Technical Debt as the Unifying Lens

The three problems above — onboarding, bug injection, regulated exposure — share a cause that the vocabulary of technical debt names precisely [25]. Debt is cost deferred into future change: a principal (the shortcut taken) and an interest (the extra effort every later change pays). Most of what Sections 3.1–3.3 describe is interest: the over-fetched graph paid for on every request, the parallel shapes synchronized on every schema change, the ambient save re-audited on every compliance review. Practitioner surveys identify architectural decisions, not code-level sloppiness, as the dominant source of such debt [28]; Slim Domain is an architectural decision aimed at exactly that source. Table 1 maps each tenet to the debt channel it closes and to the structural proxy Appendix B measures.

Tenet or rule	Debt channel closed	Interest avoided	Appendix B proxy
Flatten to the use case	Over-fetching that accretes screen by screen — Include chains, growing graphs	Latency and payload paid on every request, indefinitely	Eager-load sites: 20 → 0 per 100k lines
Keep canonical entities minimal	Entity accretion — screen members and [NotMapped] piling onto the persistence contract	Every schema change negotiated with every consumer of the entity	[NotMapped]: 627 vs. 132 per 100k; mapping attributes 12.6× lower
Inherit within a boundary; the DTO warrant test	Parallel shapes kept in sync by hand	One domain change touches N copies and the mappers between them	Mapping ceremony per line
Nouns and verbs apart	Ambient persistence — every noun a persistence actor	A cross-cutting persistence change touches hundreds of files	≈560 vs. 7 files participating in persistence
Generator contract; fitness functions	Architecture erosion and silent schema drift	Drift discovered late, at runtime, by someone else	Not yet measured (Section 11.2)

Table 1. The tenets as debt prevention: the channel each closes, the interest it avoids, and the structural proxy measured in Appendix B.

The claim is bounded in two ways, and the lens makes both precise. First, the pattern addresses inadvertent debt [26] — the shortcut taken because it was the easy path — by making the easy path the correct one; it raises the cost of a reckless shortcut, and it does nothing about debt a team takes on deliberately under deadline pressure, as no architecture can. Second, prevention is claimed by construction, not by observation: Appendix B shows the channels absent in one system and present in a legacy one, which

is a snapshot; whether absence lowers the accrual rate over a system's life is a longitudinal question, and Section 11.2 proposes the measurement.

The lens also obliges the paper to name the debt the pattern itself can carry, and to say which of it is distinctive. Schema churn is not: every persistence strategy pays it somewhere — in migrations and entity configuration under Entity Framework, in mapper layers under rich-domain designs, in SQL strings under Dapper, at read time under schema-on-read stores — and no pattern or library the authors know of avoids it on a volatile schema. What differs is where and when it is paid; Slim Domain pays it at compile time in the consuming layer, which Section 6.5 argues is the cheapest place to pay. Three vectors are distinctive and are treated as such in Section 11: the entity generator as tooling debt, acute while a team's generator is immature; dual-path drift where bypass and repository modes coexist without the fitness functions of Section 9.4; and hand-written primitives — the version predicate, caching — that can rot if they are not centralized, which is why Section 6.8 centralizes them.

With that fence in place, the lens is carried through the paper: Section 7.1 states bounded interest as a consequence, Appendix B labels its measurements as principal and interest, and the roadmap of Section 11.2 names the accrual metrics that would turn the structural claim into a longitudinal one.

---

4. Core Principles of Slim Domain

Slim Domain rests on four principles, illustrated below with C# examples from the Aegis Platform, a fictitious regulated financial-services system (account management, transactions, policy enforcement) used as the reference solution throughout this paper. Figure 5 shows the dependency structure.

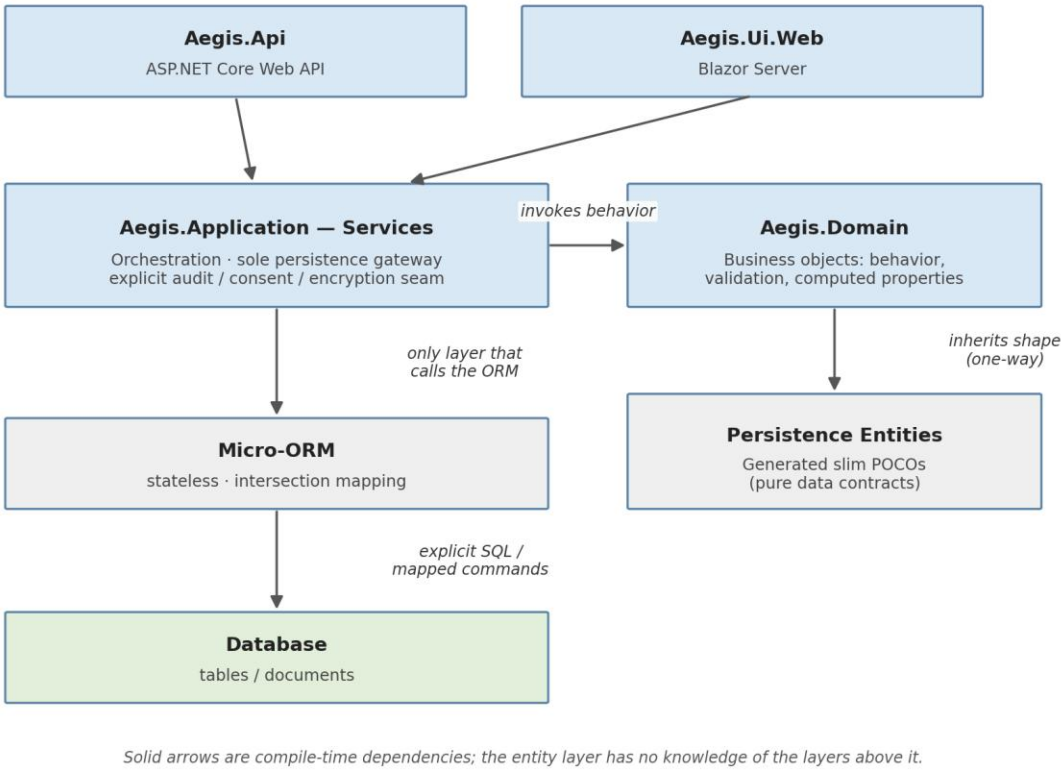


Figure 5. Slim Domain layers and compile-time dependencies in the Aegis Platform.

4.1 Slim Entities as Pure Data Contracts

The persistence model consists of minimalist plain-old CLR objects (POCOs) containing only the properties required to map to database tables or documents. These entities have no behavior, no navigation properties, no methods, and no awareness of business rules. They are typically generated from the schema. The one attribute they carry names the table: the Entity suffix that distinguishes the canonical class from the business object also means the class name is not the table name, so the generator emits [Table] on every canonical entity rather than relying on the library's name inference. (A team that prefers inference can drop the suffix instead; what matters is that the naming convention and the library's inference agree.)

```
namespace Aegis.Infrastructure.Persistence.Entities;

// Generated from the database schema. No behavior, no navigation
// properties; the one attribute names the table, because the Entity
// suffix means the class name is not the table name.
[Table("account")]
public class AccountEntity
{
    public int Id { get; set; }
    public string AccountNumber { get; set; } = string.Empty;
    public int HolderId { get; set; }
    public decimal Balance { get; set; }
    public string Status { get; set; } = AccountStatus.Pending;
    public int Version { get; set; }          // concurrency (Section 6.7)
    public DateTime CreatedUtc { get; set; }
    public DateTime ModifiedUtc { get; set; }
    public DateTime? DeletedUtc { get; set; }     // soft delete (Section 6.8)
    public string? DeletedBy { get; set; }
}

// Hand-written companion, not generated: named values for a status
// column, so no rule compares against a bare string.
public static class AccountStatus
{
    public const string Pending = "Pending";
    public const string Active = "Active";
    public const string Suspended = "Suspended";
}
```

Listing 2. A generated slim entity: the persisted shape and nothing else.

4.2 Enriched Business Objects via Inheritance or Composition

Business objects acquire the persistence shape — via inheritance, at no ceremony cost, or via composition, through clean delegation — and then add computed properties, validation logic, business methods, and richer data structures. Both approaches are first-class citizens in the pattern. The relationship is one-way: the business layer depends on the entity contract for shape, but the entity layer has no knowledge of business objects.

```
namespace Aegis.Domain;

// Acquires the persisted shape by inheritance and adds behavior.
// Inherited setters are public here, which suits modules whose fields
// are individually editable by design; emit them as protected set
// where invariants matter (Section 4.2).
public class Account : AccountEntity
{
    // Computed properties: no matching columns, never persisted.
    public bool IsOverdrawn => Balance < 0m;
    public string DisplayLabel => $"{AccountNumber} ({Status})";

    // Richer structure populated by services; also never persisted.
    public List<Transaction> RecentTransactions { get; } = new();

    public void ApplyDebit(decimal amount, DateTime nowUtc)
    {
        if (amount <= 0m)
            throw new DomainRuleException(
                "Debit amount must be positive.");
        if (Status != AccountStatus.Active)
            throw new DomainRuleException(
                "Debits require an active account.");

        Balance -= amount;
        ModifiedUtc = nowUtc;      // time is supplied, not read
    }
}
```

Listing 3. The inheritance variant: behavior, validation, and computed properties added to the persisted shape.

Where stricter encapsulation is required — for example, to prevent arbitrary mutation through inherited public setters — the composition variant wraps the entity and exposes only intention-revealing members:

```
// Composition variant: the entity is private state, so the only
// route to the balance is a method that enforces the rules.
public sealed class Account
{
    private readonly AccountEntity _data;

    public Account(AccountEntity data) => _data = data;

    public int Id => _data.Id;
    public string AccountNumber => _data.AccountNumber;
    public decimal Balance => _data.Balance;
    public string Status => _data.Status;
    public bool IsOverdrawn => _data.Balance < 0m;

    public void ApplyDebit(decimal amount, DateTime nowUtc)
    {
        if (amount <= 0m)
            throw new DomainRuleException(
                "Debit amount must be positive.");
        if (_data.Status != AccountStatus.Active)
            throw new DomainRuleException(
                "Debits require an active account.");

        _data.Balance -= amount;
        _data.ModifiedUtc = nowUtc;
    }

    public void ApplyCredit(decimal amount, DateTime nowUtc)
    {
        if (amount <= 0m)
            throw new DomainRuleException(
                "Credit amount must be positive.");
        _data.Balance += amount;
        _data.ModifiedUtc = nowUtc;
    }

    // Hands the persistence shape to the service layer for the write.
    public AccountEntity ToEntity() => _data;
}
```

Listing 4. The composition variant: the entity is private state behind a behavioral façade.

The service-side round trip — wrap on load, unwrap on save — is shown in full in Listing 5, the paper's flagship service, which uses this variant.

Choosing between the variants. Both are first-class, but the choice should be deliberate. For new work in regulated domains — or wherever invariants carry compliance weight — the recommended default is composition, because public inherited setters remain a bypass vector even under a strong review culture; a junior developer can always assign *account.Balance* directly in the inheritance variant, and only convention prevents it. Inheritance with protected setters is a strong intermediate: the generator emits protected setters, the data-access library materializes through them —

verified by test for FunkyORM, and true of Dapper and Entity Framework Core as well — and the bypass closes for every caller except the business object itself, while the inherited shape and the absence of in-process mapping are kept. Inheritance with public setters remains the lowest-ceremony choice for CRUD-heavy modules whose invariants are simple and whose fields are individually editable by design.

Because the reference domain is financial, the flagship service and every listing on its road (Listings 5, 13, and 17) use the composition variant. The attribute-road listings (6, 8, and 18) use the inheritance variant, because that is the shape intersection mapping rewards, and they are labeled as such. The inheritance variant remains the lowest-ceremony form of the pattern and the one to prefer where its precondition holds; it is not the one to copy into a ledger.

Composition does not require additional binary references beyond what inheritance already implies in most .NET scenarios, and in many cases reduces coupling compared to rich domain models that mix behavior and persistence concerns.

4.3 Service-Gated Persistence

The services layer is the only part of the system that references the data-access library or performs persistence operations. Business objects contain no persistence logic. All loads, saves, queries, and transactions are initiated and managed by service methods, which makes every persistence operation an explicit, auditable call site — and a natural place for consent checks, encryption, structured logging, and domain-event emission.

Listing 5 is the paper's flagship service: the debit a team would copy first. It is written on Dapper over a plain IDbConnection — the ecosystem's default micro-ORM, chosen here precisely because it offers none of the conveniences of the attribute road — in the composition variant, inside one transaction, with a version predicate, and with the domain rule unreachable except through ApplyDebit. Every later listing on the flagship road is a variation on this shape; the attribute-road listings follow the same steps in FunkyORM's vocabulary.

```
namespace Aegis.Application.Services;

// The flagship debit. One persistence API (Dapper over IDbConnection),
// one transaction, a compare-and-swap on the version column, and a
// domain rule that cannot be bypassed because the entity is private.
public sealed class AccountService
{
    // Column aliases bind snake_case columns to the entity under default
    // Dapper; alternatively set DefaultTypeMap.MatchNamesWithUnderscores
    // once at startup and drop the aliases.
    private const string SelectSql =
        "SELECT id, account_number AS AccountNumber, holder_id AS HolderId, " +
        "balance, status, version, created_utc AS CreatedUtc, " +
        "modified_utc AS ModifiedUtc FROM account " +
        "WHERE id = @id AND deleted_utc IS NULL";   // never act on a deleted row

    private const string UpdateSql =
        "UPDATE account SET balance = @Balance, status = @Status, " +
        "modified_utc = @ModifiedUtc, version = @Version " +
        "WHERE id = @Id AND version = @expectedVersion " +
        "AND deleted_utc IS NULL";

    private readonly IDbConnection _db;   // scoped: opened with the unit
    private readonly IAuditWriter _audit; // of work, disposed with the scope
    // IAuditWriter.WriteAsync(action, subjectId, correlationId,
    //     IDbTransaction? tx = null, string? purpose = null)
    private readonly IClock _clock;

    public AccountService(IDbConnection db, IAuditWriter audit,
                          IClock clock)
    {
        _db = db; _audit = audit; _clock = clock;
    }

    public async Task<Account> DebitAsync(int id, decimal amount,
                                          Guid correlationId)
    {
        using var tx = _db.BeginTransaction();   // disposed => rolled back

        var entity = await _db.QuerySingleOrDefaultAsync<AccountEntity>(
            SelectSql, new { id }, tx)
            ?? throw new NotFoundException(nameof(Account), id);
        var expectedVersion = entity.Version;

        var account = new Account(entity);          // wrap: rules live here
        account.ApplyDebit(amount, _clock.UtcNow);  // the only way to change Balance

        var row = account.ToEntity();               // unwrap for the write
        row.Version = expectedVersion + 1;
        int rows = await _db.ExecuteAsync(UpdateSql,
            new { row.Balance, row.Status, row.ModifiedUtc, row.Version,
                  row.Id, expectedVersion }, tx);
        if (rows == 0)
            throw new ConcurrencyConflictException(nameof(Account), id);

        await _audit.WriteAsync("Account.Debit", id, correlationId, tx);
        tx.Commit();
        return account;
```

```
    }
}
```

Listing 5. The flagship debit: one persistence API, one transaction, a compare-and-swap, and a rule that cannot be bypassed.

Four properties of this listing carry the pattern. The business object is the only route to the balance, because the entity is private state (Section 4.2). The write names its columns and its version predicate, so a concurrent debit loses cleanly instead of silently (Section 6.7). The transaction is owned by the method that opened it and is rolled back on any exception by disposal. And the audit write shares the transaction, so an audited debit and its audit record commit or roll back together — which is what "explicit, auditable call site" has to mean in a regulated system.

4.4 Use-Case-Shaped Reads, and the Attribute Road's Write Convenience

The fourth principle concerns reads: every query returns a shape carrying only what the operation uses — a canonical entity when the operation acts on one row, otherwise a detail entity, a view, or a projection (Sections 2.6 and 6.3). Listing 5 selects a named column list into the canonical shape; Listing 21 selects a projection type that cannot name a forbidden column. The ratio of fields loaded to fields used is the measure, and it is library-independent.

Where the data-access library maps only the intersection of database columns and object properties — in both directions, with no attributes — the inheritance variant becomes almost free: the enriched business object itself can be handed to the write, and properties present on the object but absent from the table are ignored. FunkyORM does this by default (Table 2); most micro-ORMs do it on reads and require an attribute on writes. This is a convenience of the attribute road, not a requirement of the pattern — the flagship service of Listing 5 persists without it, naming its columns — and Section 6.4 covers the cases where conventions need help.

```
// Attribute road (FunkyORM): because mapping is the intersection of
// columns and properties in both directions, the enriched business
// object itself can be handed to the write.
var account = await _provider.GetAsync<Account>(42);

account.ApplyDebit(150.00m, _clock.UtcNow);

// UpdateAsync writes only the intersection of Account's properties
// and the account table's columns. IsOverdrawn, DisplayLabel, and
// RecentTransactions have no matching columns and are ignored: no
// [NotMapped] attributes, no DTO projection, no mapper configuration.
await _provider.UpdateAsync(account);
```

Listing 6. Attribute road, inheritance variant: intersection mapping lets the enriched object be written with no configuration — and with no predicate (see Listing 18 for the guarded write).

The same debit on the view road names the tracked type explicitly and lets the library's concurrency token supply the predicate:

```
// View road (Entity Framework Core): the tracked type is the slim
// entity, the behavioral object wraps it, lazy loading is off, and
// SaveChanges is confined to the service. Version is configured as a
// concurrency token, so EF emits WHERE version = @original — but a
// token only conflicts if it changes, so the service changes it.
public async Task<Account> DebitAsync(int id, decimal amount,
                                      Guid correlationId)
{
    var entity = await _db.Accounts            // DbSet<AccountEntity>
        .SingleOrDefaultAsync(a => a.Id == id)   // no Include, no graph
        ?? throw new NotFoundException(nameof(Account), id);

    var account = new Account(entity);
    account.ApplyDebit(amount, _clock.UtcNow);
    entity.Version++;   // the compare-and-swap, in EF's vocabulary

    // The audit row is part of the same SaveChanges, so the debit and
    // its audit record commit or roll back together.
    _db.AuditEvents.Add(AuditEvent.For("Account.Debit", id, correlationId));

    try   { await _db.SaveChangesAsync(); }
    catch (DbUpdateConcurrencyException)
    {
        throw new ConcurrencyConflictException(nameof(Account), id);
    }
    return account;
}

// Configuration: modelBuilder.Entity<AccountEntity>()
//     .Property(a => a.Version).IsConcurrencyToken();
// A store-generated rowversion/xmin token is the alternative: EF then
// updates it for you and the increment above is unnecessary.
// Read-side shapes on this road are keyless types mapped to views:
//   modelBuilder.Entity<AccountSummary>().HasNoKey().ToView("account_summary");
// read with AsNoTracking(); the canonical entity carries no navigation
// properties, so there is no graph to Include.
```

Listing 7. The view road (Entity Framework Core): the slim entity is the tracked type, the behavioral object wraps it, and SaveChanges is confined to the service.

Three roads, one shape: load the slim entity, wrap or inherit it, invoke behavior, write with a predicate. Listing 5 carries the predicate in its SQL; Listing 7 changes the token so that Entity Framework's own predicate can conflict; on the attribute road the predicate lives in the stored procedure of Listing 18, because Listing 6's UpdateAsync has none — it is the convenience, not the guarded write. On the view road one further caution applies: a tracked entity mutated anywhere is persisted by the next SaveChanges, so the structural guarantee of the other two roads becomes a convention held by the fitness functions of Section 9.4.

These four principles work together to produce systems that are easy to reason about, resistant to common classes of bugs, and straightforward for developers of varying experience to implement correctly.

4.5 How the Principles Map to SOLID

The pattern predates its name (and once carried "Architecture" in it, a word Section 2 explains it does not deserve), and its shape owes a visible debt to the SOLID principles [18], which informed how it came about. The correspondence is direct:

- Single responsibility. Slim entities carry the persistence contract; business objects carry behavior; services orchestrate and gate persistence. Three concerns, three homes — and because the generator owns the first, the concerns cannot quietly merge.
- Open/closed. Detail entities (Section 6.3) extend read-side shapes without touching canonical contracts, and schema-derived interfaces (Section 6.8) let a newly added table acquire centralized behavior with no modification to the code that implements it. This is ordinary polymorphism plus generation rather than a novel mechanism; what the pattern adds is the location of change — the schema and the generator, not the service code.
- Liskov substitution. A hazard as much as a satisfaction, and the paper says so. Because entity bases carry no behavior, a business object substitutes for its entity contract without altering any inherited behavior — there is none to alter. But the same substitutability means an API that accepts the entity will accept the business object and persist it whether or not ApplyDebit ran. The pattern does not resolve that by appeal to the principle; it resolves it structurally — the composition variant makes the entity private, and the service gate is the only write path (Section 4.3).

- Interface segregation. Cross-cutting consumers depend on narrow, schema-derived interfaces — IAuditable, ISoftDeletable — rather than on concrete entities with dozens of unrelated members (Section 6.8).
- Dependency inversion. Services depend on provider and repository abstractions, enforceable at build time (Sections 9.3–9.4). The pattern's one deliberate exception — business objects depend on the entity shape — is a considered trade examined in Sections 6.5 and 10.

4.6 The Generator Contract

Entity generation is load-bearing in this pattern (Table 3) and in the cross-cutting technique of Section 6.8, and .NET teams carry long memories of designer models, T4 templates, and scaffold-once tools that could not be regenerated safely. A generator the pattern can rely on must therefore satisfy a contract, stated here so that any implementation — an in-house tool, a script, a template — can be judged against it:

- Input. The live schema (INFORMATION_SCHEMA or the engine's catalog) or a committed schema snapshot; never a hand-maintained model.
- Output. One partial class per table, in a file the generator owns outright; hand-written members live in a sibling partial file the generator never touches. Regeneration overwrites its own files and nothing else. [Table] is emitted on every canonical class, because the Entity suffix convention means the class name never matches the table name; [Column] only where a column name deviates from convention.
- Renames. A renamed column is a removal and an addition; the compiler reports every affected member. There is no silent remap.
- Column-group suggestions. Recurring groups are written to a report, not applied. A committed approval file lists the interfaces and base classes the team has accepted; only approved groups are emitted, and a new suggestion fails nothing until a human acts on it.
- Drift. A continuous-integration step regenerates into a scratch location and fails the build on any diff, so the schema and the entities cannot diverge unnoticed (Section 9.4).

- Escape hatch. A table can be excluded and a canonical entity pinned to a hand-written class, so a trial can begin with hand-written POCOs and adopt generation table by table.

A team whose generator meets this contract can adopt Section 6.8 from the paper alone. A team without one can still practice flattened reads, minimal canonical entities, and the service gate with hand-written entities, and add generation later.

---

5. Micro-ORMs vs. Monolithic ORMs — A Strong Recommendation

One of the most consequential decisions in any .NET data-access strategy is the choice of ORM. Slim Domain is designed around micro-ORMs and deliberately avoids the stateful, heavy-configuration, event-sequence-heavy model of Entity Framework and similar monolithic ORMs.

5.1 Why Micro-ORMs Align with Slim Domain's Goals

Entity Framework and similar full ORMs provide powerful features — change tracking, identity maps, lazy loading, global query filters, and automatic optimistic concurrency. These features come at a cost:

- Hidden state and side effects that are difficult to reason about
- N+1 query risks from navigation properties
- Complex event sequences (SaveChanges, DetectChanges, and related machinery) that must be understood to avoid subtle bugs
- Heavy configuration (ModelBuilder, conventions, interceptors) that increases the learning curve
- Proxy generation and identity management that can interfere with serialization, testing, and distributed systems

Micro-ORMs offer a different trade-off: explicit, stateless, predictable data access with minimal abstraction. Combined with service-gated persistence and intersection mapping, they deliver no hidden state, no N+1 risk from navigation properties (there are none), simple and auditable query logic, near-zero configuration, strong performance characteristics, and a straightforward evolution path to event sourcing or CQRS.

5.2 The Candidates

- Dapper — the de facto standard; explicit SQL with fast, simple materialization [9].
- FunkyORM — lambda-powered LINQ-style querying built around intersection mapping; maintained by Funcular Labs (see the disclosure in Section 1.4) [12].
- RepoDB — a hybrid of micro-ORM speed and fuller feature set, with strong bulk-operation support [13].
- PetaPoco — extremely lightweight and proven in high-volume systems [14].
- SqlKata + Dapper — a query builder paired with a micro-ORM; keeps SQL explicit while providing fluent construction [15].
- LinqToDB (lightweight configuration) — the broadest LINQ support short of Entity Framework, with far less overhead [16].
- ServiceStack.OrmLite — simple, fast, and well-suited to service-oriented architectures; commercially licensed above free-tier quotas [17].

5.3 Feature Comparison Matrix

Table 2 compares the candidates on the capabilities most relevant to Slim Domain. All seven are stateless by design — none carries a change tracker, identity map, or proxy layer — which is precisely why they qualify. The matrix therefore focuses on where they differ.

Capability	Dapper	FunkyORM 3.12.1†	RepoDB	PetaPoco	SqlKata + Dapper	LinqToDB	OrmLite
Primary query interface	Raw SQL	LINQ lambdas	Fluent ops + expressions	SQL templates	Fluent builder	Full LINQ	Typed expressions
Lambda / LINQ predicates	No	Yes	Partial	No	No	Yes	Yes
Automatic intersection mapping (both directions, no attributes)*	Reads only	Yes	Partial	Partial	Reads only	Partial	Partial
Mapping attribute system	Contrib's own	Standard Data-Annotations	Own set	Own set	n/a	Own set	Own set
Engines with native SQL generation	n/a (hand-written SQL)	MSSQL, PostgreSQL, MySQL, SQLite	MSSQL, PostgreSQL, MySQL, SQLite	Many	Many (compilers)	Many	Many

Capability	Dapper	FunkyORM 3.12.1†	RepoDB	PetaPoco	SqlKata + Dapper	LinqToDB	OrmLite
Cross-table flattening without raw SQL	No	Yes ([Remote-Property], [RemoteKey])	No	No	Via builder joins	Yes (LINQ joins)	Partial (typed joins)
JSON column query helpers	Via raw SQL	Yes ([JsonPath], 4 engines)	Via raw SQL	Via raw SQL	Via raw fragments	Partial (extensions)	Via raw SQL
View-replacing computed / aggregate attributes	No	Yes (4 attribute types)	No	No	No	Partial ([Expression-Method])	No
Stored procedure execution	Yes	Yes‡	Yes	Yes (via SQL)	Via Dapper	Yes	Yes
Bulk operations	Via extensions	No	Yes (flagship)	Partial	No	Yes (BulkCopy)	Partial
Relative maturity and community	Very large	Emerging	Moderate	Mature	Moderate	Mature	Mature (commercial)

Table 2. Micro-ORM feature comparison, compiled from project documentation in June 2026; FunkyORM column refreshed against release 3.12.1 in October 2026.

** Materialization (reading) ignores unmatched result columns in most micro-ORMs; ignoring unmatched object properties on writes typically requires an attribute ([Computed], [Ignore], [ResultColumn], [NotColumn], or equivalent). FunkyORM applies intersection mapping in both directions by default, which is what allows enriched business objects to be persisted without configuration.*

† FunkyORM rows reflect release 3.12.1 (October 2026); stored-procedure execution shipped in 3.7.0. Releases 3.10 through 3.12 tightened query-translation correctness — untranslated operators now throw rather than mistranslate, and null comparisons in WHERE follow C# semantics — and the project documents its remaining divergences; readers should verify current capabilities against the repository [12].

‡ Stored-procedure support is bound by engine capability everywhere, not only in FunkyORM: full support on SQL Server and MySQL; on PostgreSQL, procedures return scalars and row counts via CALL while tabular results are idiomatically exposed as functions; SQLite has no stored procedures at all.

A word on the rows FunkyORM wins. Bidirectional intersection mapping, view-replacing attributes, JSON helpers, and cross-table flattening are what make the attribute road low-ceremony; they are not what makes the pattern. The flagship service (Listing 5) and the view-road write (Listing 7) obey every tenet with none of them. The table is a guide to on-ramps, not a definition.

The recommendation is not "never use Entity Framework." For certain greenfield projects with complex reporting needs, or for teams already deeply invested in the EF Core ecosystem, it may be the right tool. For the majority of line-of-business and regulated systems, however, a micro-ORM is the better fit for the pattern's goals — explicit, stateless persistence with no hidden pipeline to reason about — a judgment about mechanism rather than a measured result. Entity Framework teams can practice

the pattern's central habit without changing libraries; Section 2.6 describes how. Notably, the historical feature advantages of full ORMs are narrowing: as Table 2 shows, capabilities such as JSON querying, computed projections, and stored-procedure execution are now available in the micro-ORM ecosystem without adopting a change tracker.

---

6. Advanced Techniques in Practice

The most common objections to Slim Domain are practical: How do you load related data without navigation properties? What about transactions? What about shapes that span tables? This section answers each with working code. Listings in this section are labeled by road. Flagship-road listings use Dapper, the composition Account of Listing 4, and the connection and transaction of Listing 5; attribute-road listings use FunkyORM syntax and the inheritance Account of Listing 3, which is the shape intersection mapping rewards. Table 2 indicates which capabilities have equivalents in the other micro-ORMs, and the techniques themselves are portable.

6.1 Loading Object Graphs Without Navigation Properties

A clarification first: navigation properties are not prohibited in Slim Domain — they are simply not the *organizing principle* for data retrieval. Business objects may carry collections of related business objects (Listing 3's RecentTransactions); what the pattern rejects is making such properties implicit triggers for data access, as lazy loading does. Retrieval remains explicit, and three tactics cover the practical cases.

Tactic 1 — Explicit composition. The service issues one narrow query per shape and assembles the result. This is the workhorse approach:

```
// Explicit composition: two narrow queries, no joins, and no
// duplicated parent data on the wire.
public async Task<Account?> GetWithRecentActivityAsync(int id)
{
    var account = await _provider.GetAsync<Account>(id);
    if (account is null) return null;

    var recent = _provider.Query<Transaction>()
        .Where(t => t.AccountId == id)        // RemoteLink/foreign key
        .OrderByDescending(t => t.PostedUtc)
        .Take(20)
        .ToList();

    account.RecentTransactions.AddRange(recent);
    return account;
}
```

Listing 8. Explicit graph composition (attribute road, inheritance Account of Listing 3): each shape is fetched once, in full, with no row duplication.

This approach is frequently more efficient on the wire than an ORM's eager-loading strategy. An Entity Framework .Include() emits a join that repeats every parent column on every child row (or, with split queries, issues comparable multiple queries with added round-trip coordination). Explicit composition retrieves each shape exactly once. It also makes the cost of every load visible in code review — there is no innocuous property access that silently becomes a query.

Tactic 2 — Attribute-defined projections. Where the goal is a read-side shape rather than a graph of behavioral objects, view-replacing attributes (Section 6.3) project related data — computed expressions, child aggregates, JSON extractions, even child collections as JSON — directly onto a detail entity in a single query.

Tactic 3 — Stored procedures. Where the database team owns complex retrieval logic, stored procedures return shaped result sets through the same mapping pipeline:

```
// Stored procedure execution (FunkyORM 3.7 and later): result sets
// map through the same pipeline as Query<T>. Parameters may be
// anonymous objects, typed classes, tuples, or SqlParam objects
// (the latter supporting output parameters).
var statement = _provider.ExecProcedure<AccountStatementRow>(
    "sp_get_account_statement",
    new { account_id = 42, months = 3 });

int flagged = _provider.ExecScalar<int>(
    "sp_count_flagged_accounts",
    ("@risk_tier", "high"));            // tuple syntax
```

Listing 9. Stored-procedure execution: shaped retrieval through the standard mapping pipeline (FunkyORM 3.7 and later).

Other micro-ORMs offer their own tactics for the same problem: Dapper's QueryMultiple and multi-mapping APIs compose graphs from a single batch; RepoDB provides QueryMultiple; LinqToDB expresses joins and eager loads in full LINQ (LoadWith); OrmLite offers typed join builders and reference loading. The pattern's requirement is only that retrieval be explicit — every one of these tactics satisfies it.

6.2 Transactions: Implicit, Atomic, Named, and Multi-Call

Stateless data access does not mean transactionless data access. Three usage levels cover the practical range, in increasing order of scope:

```
// Implicit: every standalone operation is atomic. Each call
// receives its own pooled connection and commits on success —
// no ceremony, no shared state.
var id = _provider.Insert<Account, int>(newAccount);
_provider.Update(account);
```

Listing 10. Implicit atomicity: standalone operations commit independently, each on its own pooled connection.

```
// Atomic multi-call: all operations share one connection and
// commit or roll back together. Within a transaction, operations
// must be awaited sequentially (an ADO.NET constraint that the
// provider enforces with a clear exception).
_provider.BeginTransaction();
try
{
    await _provider.UpdateAsync(source);          // debit
    await _provider.UpdateAsync(destination);     // credit
    await _provider.InsertAsync<TransferRecord, int>(record);
    _provider.CommitTransaction();
}
catch
{
    _provider.RollbackTransaction();
    throw;
}
```

Listing 11. Atomic multi-call transaction: explicit begin/commit/rollback around multiple operations.

```
// Named: Commit/Rollback act only when the supplied name matches
// the active transaction. Composed helpers cannot accidentally
// commit a transaction they do not own.
_provider.BeginTransaction("transfer");
try
{
    await DebitAsync(source, amount);       // helper logic may run
    await CreditAsync(destination, amount); // inside the same scope

    _provider.CommitTransaction("transfer"); // only the owner commits
}
catch
{
    _provider.RollbackTransaction("transfer");
    throw;
}
```

Listing 12. Named transaction: commit and rollback are guarded by name, protecting ownership across composed calls.

The named form deserves emphasis in a service-gated architecture: when service methods compose (a TransferService calling helpers that also touch the database), the name guard ensures that only the method that opened the transaction can complete it — an ownership rule enforced by the library rather than by convention. The provider also enforces sequential awaiting within a transaction, converting a notoriously cryptic ADO.NET misuse ("a reader is already associated with this connection") into an immediate, clearly worded exception. Outside transactions, every operation receives its own pooled connection, so concurrent reads in environments such as Blazor Server require no special handling.

In Listing 12 the helpers share the service's provider instance; that is what places them inside the transaction, and two rules make it safe. First, a provider that can hold a transaction is registered with a scoped lifetime — per request in ASP.NET Core, per circuit in Blazor Server, per unit of work in a background service — and never as a singleton, because a singleton with transaction state is a cross-request transaction bug of exactly the class the pattern claims to remove. Second, helpers receive the scoped instance through injection rather than opening connections of their own; a helper that creates its own provider is not in the transaction, commits on its own, and bypasses the ownership guard. On a library without ambient transactions the same discipline is explicit: the transaction is passed as a parameter, and a helper that takes it cannot commit it.

Cross-entity consistency — the reason tactical domain-driven design loads an aggregate — is handled by the service, and it deserves to be shown rather than asserted. A transfer is the reference domain's consistency boundary: two balances and a ledger line either commit together or not at all, under rules that span both accounts.

```
// A consistency boundary spanning two accounts, on the flagship API.
// Only TransferAsync can commit; the helpers take the transaction and
// cannot. The spanning rules live in one domain type. (In the reference
// solution, Listing 5's DebitAsync is the same two helpers under their
// own transaction.)
private readonly ILedgerWriter _ledger;   // injected beside _audit

public async Task TransferAsync(int fromId, int toId, decimal amount,
                                Guid correlationId)
{
    using var tx = _db.BeginTransaction();
    var now = _clock.UtcNow;

    // Load in key order so two opposite transfers cannot deadlock.
    var first = await LoadAsync(Math.Min(fromId, toId), tx);
    var second = await LoadAsync(Math.Max(fromId, toId), tx);
    var (source, target) = fromId < toId ? (first, second)
                                         : (second, first);

    TransferRules.Check(source.Account, target.Account, amount);

    source.Account.ApplyDebit(amount, now);
    target.Account.ApplyCredit(amount, now);

    await WriteAsync(source, tx);
    await WriteAsync(target, tx);
    await _ledger.AppendAsync(fromId, toId, amount, correlationId, tx);
    await _audit.WriteAsync("Account.Transfer", fromId, correlationId, tx);

    tx.Commit();
}

private sealed record Loaded(Account Account, int ExpectedVersion);

private async Task<Loaded> LoadAsync(int id, IDbTransaction tx)
{
    var entity = await _db.QuerySingleOrDefaultAsync<AccountEntity>(
        SelectSql, new { id }, tx)                 // the SQL of Listing 5
        ?? throw new NotFoundException(nameof(Account), id);
    return new Loaded(new Account(entity), entity.Version);
}

private async Task WriteAsync(Loaded loaded, IDbTransaction tx)
{
    var row = loaded.Account.ToEntity();
    row.Version = loaded.ExpectedVersion + 1;
    int rows = await _db.ExecuteAsync(UpdateSql,
        new { row.Balance, row.Status, row.ModifiedUtc, row.Version,
              row.Id, expectedVersion = loaded.ExpectedVersion }, tx);
    if (rows == 0)
        throw new ConcurrencyConflictException(nameof(Account), row.Id);
}

// Domain: rules that span the pair, impossible to skip from a helper.
public static class TransferRules
{
    public static void Check(Account source, Account target,
```

```
                              decimal amount)
    {
        if (source.Id == target.Id)
            throw new DomainRuleException(
                "Cannot transfer to the same account.");
        if (source.Balance < amount)
            throw new DomainRuleException(
                "Transfers may not overdraw the source.");
    }
}
```

Listing 13. A consistency boundary: two accounts, one transaction, spanning rules in one domain type, and helpers that cannot commit.

Three things distinguish this from Transaction Script. The rules that span the pair live on a domain type, not in the service; the per-account rules live on the business objects; and the service contributes only ordering, the transaction, and the writes. The aggregate that tactical domain-driven design would load to protect this invariant is replaced by a transaction plus a version predicate per row — which is what the aggregate's concurrency token was doing anyway.

6.3 Non-Canonical Entities: Detail Classes, Views, and Multi-Table Shapes

Slim Domain distinguishes *canonical entities* — the generated, one-table persistence contracts of Section 4.1 — from *detail entities* (non-canonical): read-mostly classes whose shape spans related tables, aggregates children, or extracts semi-structured data. Detail entities answer the reporting and display needs that tempt teams to bloat their canonical models or proliferate database views.

```
// Detail entity (non-canonical): a read-mostly projection that
// reaches across related data via attribute-defined SQL — no
// database view, no stored procedure, no raw SQL. Sealed, and marked
// IReadModel so no write path accepts it (Section 6.3).
[Table("account")]
public sealed class AccountSummary : AccountEntity, IReadModel
{
    [SqlExpression("COALESCE({Balance}, 0)")]
    public decimal EffectiveBalance { get; set; }

    [SubqueryAggregate(typeof(TransactionEntity),
        nameof(TransactionEntity.AccountId), AggregateFunction.Count)]
    public int TransactionCount { get; set; }

    [JsonPath("metadata", "$.riskTier")]
    public string RiskTier { get; set; }

    [JsonCollection(typeof(TransactionEntity),
        nameof(TransactionEntity.AccountId),
        Columns = new[] { "Amount", "PostedUtc" },
        OrderBy = "PostedUtc")]
    public string RecentActivityJson { get; set; }
}

// Filterable like any entity, including on the projected members.
// AccountSummary inherits ISoftDeletable with the canonical shape, so
// the read states its soft-delete intent (or goes through the service
// base's QueryActive, Listing 20, which states it once).
var risky = _provider.Query<AccountSummary>()
    .Where(a => a.DeletedUtc == null
             && a.RiskTier == "high" && a.TransactionCount > 100)
    .ToList();
```

Listing 14. A detail entity: view-replacing attributes project computed, aggregated, and JSON-derived members — queryable and filterable — without a database view.

Every ORM in Table 2 can map an entity to a database view, so the multi-table shape itself is universally achievable — at the cost of maintaining the view as schema. The differences lie in how much can be expressed in code instead: FunkyORM's four view-replacing attributes ([JsonPath], [SqlExpression], [SubqueryAggregate], [JsonCollection]) define the projection declaratively on the class, across all four supported engines; LinqToDB approaches the same ground through LINQ projections and [ExpressionMethod] computed members; Dapper achieves arbitrary shapes through hand-written SQL with multi-mapping; RepoDB, PetaPoco, and OrmLite generally rely on raw SQL or database views for spanning shapes. Detail entities also carry an architectural benefit: they keep the canonical entity honest. Reporting concerns accrete on purpose-built read models rather than on the persistence contract that business objects inherit.

A detail entity that inherits the canonical shape is, to the compiler, also a writable entity. Three guards close that. Detail types are sealed and carry the IReadModel marker that the generator emits (Listing 14 shows it on AccountSummary; Listing 19 declares it); on the attribute road the service base refuses IReadModel instances on its write methods (Listing 20); and an analyzer rule — described in Section 9.4 and not shown, because it is a Roslyn rule rather than an architecture test — fails the build on any insert or update whose type argument is a read model. Where the data-access library offers a read-only entity marker of its own — Entity Framework Core's keyless entity types (HasNoKey) serve — it belongs on every detail type and supersedes the runtime guard; FunkyORM offers none at the time of writing, so on the attribute road the service base and the architecture test carry the rule. A projection should never be a legal write.

Cross-table flattening attributes ([RemoteProperty], [RemoteKey], and equivalents in Table 2) deserve an explicit bridge to the pattern's principles, because they may superficially resemble the navigation properties the pattern avoids. The difference is structural: a navigation property is an open-ended graph edge on the canonical entity — an invitation for any consumer to traverse, and for the ORM to load, an unbounded object web. A remote property is a closed, declarative projection of a single value (or key) onto a detail entity, resolved as a join only in queries that use it. Canonical entities stay pure; rich read-side shapes live on purpose-built detail classes; and no query pays for a join it did not ask for. The pattern can therefore adopt these advanced micro-ORM capabilities aggressively without compromising its stance on retrieval explicitness.

6.4 Intersection-Mapping Exceptions and Data Annotations

Intersection mapping by convention raises a fair question: what about the cases where conventions are wrong — a table named differently from its class, a column that cannot be renamed, a property that coincidentally matches a column it must not map to? The answer is unremarkable by design: standard System.ComponentModel.DataAnnotations attributes handle the exceptions, exactly as they would elsewhere in .NET. One attribute is a rule rather than an exception: because canonical classes carry the Entity suffix, their class name is never the table name, so [Table] appears on every canonical entity (Listings 2, 19, and 21) and on every detail type that maps to a table.

```
// Conventions cover the common case; standard DataAnnotations
// cover the exceptions.
[Table("tbl_acct")]                 // always present (Entity suffix);
                                    // here it also absorbs a legacy name
public class AccountEntity
{
    [Key]
    [Column("acct_id")]             // column-name mismatch
    public int Id { get; set; }

    [NotMapped]                     // coincidental name collision
    public string WorkingNotes { get; set; }

    public decimal Balance { get; set; }
}
```

Listing 15. Mapping exceptions: standard attributes cover name mismatches and exclusions; conventions cover everything else.

The distinction from attribute-heavy mapping cultures is one of frequency, not capability: under intersection mapping, attributes are reserved for genuine exceptions rather than required as ceremony on every member. FunkyORM accepts the standard DataAnnotations attribute set; the other micro-ORMs in Table 2 offer equivalent control through their own attribute families ([Computed], [Ignore], [ResultColumn], [NotColumn], [Alias]) with modestly more ceremony.

6.5 Inheritance Coupling: A Deeper Look

The most substantive structural criticism of Slim Domain is that business objects inherit the persisted shape, coupling the domain layer to the schema. The criticism deserves a careful answer rather than a dismissal, because the coupling is real — the pattern's claim is that it is the right coupling, made visible.

Schema-change ripple. When the schema changes and entities are regenerated, affected business objects fail to compile. This is the intended behavior: drift between the database and the code is surfaced at build time, localized to the members that changed, rather than discovered at runtime as a mapping exception or a silently null column. Teams with volatile schemas feel this cost more often — and benefit from the visibility more often. The discipline required is that entity regeneration be tooled, versioned, and treated as a reviewed contract change (Table 3, row 1). This churn is not peculiar to the pattern — every persistence strategy pays it somewhere (Section 3.4); what the pattern chooses is to pay it earliest, where the compiler can see it.

Fragile-base-class concerns. Classic fragile-base-class problems arise when base classes carry behavior whose changes silently alter subclass semantics. Slim entities

carry no behavior — they are property bags — so the failure mode reduces to member addition and removal, which the compiler reports. The one genuine hazard is a regenerated property whose name collides with a business-object member; the compiler flags shadowing only when the signatures differ, so naming review on regeneration diffs remains a human responsibility.

Serialization leakage. Because business objects are entities (in the inheritance variant), serializing one exposes every persisted property — a data-exposure hazard at API boundaries. The Aegis structure addresses this by rule, not by hope: API layers serialize DTOs from Aegis.Shared.Contracts, never domain objects. The same rule exists for the same reason in classic n-tier; Slim Domain neither worsens nor solves it, though the explicit service seam is a natural place to enforce the projection.

Multi-consumer schemas. Where several applications consume one database, inheritance couples each domain layer to a shared schema's evolution. Mitigations are conventional: per-consumer detail entities (Section 6.3) over shared canonical tables, database views as stable contracts, or the composition variant (Listing 4), which confines schema knowledge to a wrapped private member and narrows the published surface. For genuinely divergent consumers, the pattern also does not preclude separate slim persistence models per bounded context — each generated from the same schema, each owned by its context — which is the conventional domain-driven answer to multi-consumer pressure, at the cost of running the generator once per context.

When the coupling cost is judged too high for a given context, the composition variant remains a first-class citizen of the pattern — the trade is a modest amount of delegation ceremony for full control over the exposed surface. What the pattern declines to do is reintroduce a mapping layer between two nearly identical shapes as a matter of course; that ceremony is reserved for the boundaries where it earns its keep.

6.6 Testing the Layers

The pattern's separation produces a clean testing pyramid. Business objects carry behavior but no persistence, so the bulk of domain testing requires no database, no mocks, and no fixtures:

```
// Business-object tests are pure: no database, no mocks, no fixtures.
[Fact]
public void ApplyDebit_rejects_inactive_accounts()
{
    var account = new Account(new AccountEntity
        { Status = AccountStatus.Pending, Balance = 100m });

    Assert.Throws<DomainRuleException>(
        () => account.ApplyDebit(25m, DateTime.UtcNow));
    Assert.Equal(100m, account.Balance);   // invariant held
}
```

Listing 16. Business-object tests are pure unit tests: construct, act, assert.

Services compose domain behavior with persistence, and two seams serve their tests. Where a true unit test is wanted, the thin repository interfaces of Section 9.3 accept ordinary test doubles. More often, an integration-style test against a real engine is both easier and more honest, and an in-memory SQLite database makes it cheap on any road — a Microsoft.Data.Sqlite connection under Dapper for the flagship service, or an engine-portable provider on the attribute road — exercising real SQL, real transactions, and the real version predicate with no infrastructure:

```
// Service tests: substitute the connection. An in-memory SQLite
// database gives the flagship service a real engine — real SQL, real
// transactions, a real version predicate — with no server and no mocks.
using var db = new SqliteConnection("Data Source=:memory:");
await db.OpenAsync();
await db.ExecuteAsync(AccountSchema.CreateSql);        // small DDL helper
await db.ExecuteAsync(
    "INSERT INTO account (id, account_number, holder_id, balance, " +
    "status, version, created_utc, modified_utc) " +
    "VALUES (1, 'A-1', 7, 100, 'Active', 0, @now, @now)",
    new { now = DateTime.UtcNow });

var service = new AccountService(db, new FakeAuditWriter(),
                                 new FixedClock(2026, 10, 1));

await service.DebitAsync(1, 25m, Guid.NewGuid());

Assert.Equal(75m, await db.ExecuteScalarAsync<decimal>(
    "SELECT balance FROM account WHERE id = 1"));
Assert.Equal(1, await db.ExecuteScalarAsync<int>(
    "SELECT version FROM account WHERE id = 1"));
```

Listing 17. A service test against an in-memory engine: real SQL and mapping, no mocks, no test database server.

Because the flagship holds no hidden state — a connection, a transaction, and nothing tracked — neither seam fights a context: there is no change tracker to detach, no identity map to reset between tests, and no in-memory provider whose semantics diverge from the production engine's, a documented hazard of Entity Framework's InMemory provider. Note that in-memory SQLite still differs from a production engine

in dialect edge cases; suites that depend on engine-specific SQL should also run against the production engine in continuous integration.

6.7 Optimistic Concurrency with Intersection Mapping

Full ORMs automate optimistic concurrency; the pattern's position is that the mechanism is small enough to own explicitly — one version column, one increment, one WHERE predicate. Listing 5 shows it on Dapper as a single guarded UPDATE; Listing 7 shows Entity Framework Core supplying the same predicate from a concurrency token. On the attribute road the picture is less tidy today: FunkyORM 3.12.1 has no predicate-update primitive, so the compare-and-swap is expressed as a stored procedure, which the provider executes inside the ambient transaction:

```
// Attribute road (inheritance Account of Listing 3): FunkyORM 3.12.1 has
// no predicate-update primitive, so the compare-and-swap is a
// single-statement stored procedure, executed through the provider inside
// the ambient transaction.
//
//   CREATE PROCEDURE account_update_cas (@id, @balance, @status,
//       @modified_utc, @expected_version) AS
//     UPDATE account SET balance = @balance, status = @status,
//         modified_utc = @modified_utc, version = version + 1
//     WHERE id = @id AND version = @expected_version;

var expectedVersion = account.Version;        // captured on load
account.ApplyDebit(amount, _clock.UtcNow);

int rows = await _provider.ExecNonQueryAsync("account_update_cas",
    new { id = account.Id, balance = account.Balance,
          status = account.Status, modified_utc = account.ModifiedUtc,
          expected_version = expectedVersion });

if (rows == 0)   // someone else won the race: surface it
    throw new ConcurrencyConflictException(nameof(Account), account.Id);
```

Listing 18. The compare-and-swap on the attribute road: a stored procedure carries the predicate until the library offers one.

This quantifies a trade-off raised fairly by critics: the micro-ORM approach does require writing this update by hand (or via a helper in the service base class), where Entity Framework would generate it from a [Timestamp] annotation. The delta is roughly a dozen explicit lines per aggregate that needs it — paid only where contention is real — in exchange for a mechanism that appears in code review, can be unit-tested directly (two competing updates; the loser must throw), and never silently degrades when an entity is detached from its tracking context. Where rowversion/xmin-style engine columns are preferred over an integer, the same predicate applies unchanged. The

honest cost on the attribute road is the procedure: until a predicate-update primitive exists, the guarded write is the one operation that leaves the LINQ surface.

6.8 Schema-Derived Interfaces: Centralizing Cross-Cutting Entity Concerns

Most schemas repeat themselves. Audit columns — created and modified timestamps and principals — appear on most tables; soft-delete markers and version columns on many. Untreated, repetition in the schema becomes repetition in every service that must remember to stamp CreatedBy or filter deleted rows, and a single forgotten stamp in a regulated system is an audit finding. This is the cross-cutting-concern problem as it manifests in a slim persistence model, and it is where the SOLID mapping of Section 4.5 earns its keep.

The recommended mechanism is to make the entity generator schema-aware. Beyond emitting one class per table, the generator scans the schema for the distinct recurring column groups and emits one interface per group — and, where a group is near-universal or strictly nested, optionally a common abstract base class. Section 4.6 specifies such a generator. In the deployments behind this paper the interfaces were derived from the same schema analysis by hand, with a human approving each grouping — an approval step that matters, because column-name coincidence is not semantic identity — and the contract in Section 4.6 is written so that a tool can take over the scan without changing what the team approves.

```
// Generated from schema analysis: the audit column group
// (created_utc, created_by, modified_utc, modified_by) recurs on
// 41 of 58 tables; the soft-delete group on 12; row versions on 23.
// One interface per recurring group:
public interface IAuditable
{
    DateTime CreatedUtc { get; set; }
    string CreatedBy { get; set; }
    DateTime ModifiedUtc { get; set; }
    string ModifiedBy { get; set; }
}

public interface ISoftDeletable
{
    DateTime? DeletedUtc { get; set; }
    string? DeletedBy { get; set; }
}

public interface IVersioned
{
    int Version { get; set; }     // pairs with Section 6.7
}

// Marker emitted on detail (read-model) types and never on canonical
// entities; the service base refuses to write anything that carries it.
public interface IReadModel { }

// Near-universal groups may also be emitted as an abstract base,
// so the members themselves are written once:
public abstract class AuditableEntity : IAuditable
{
    public DateTime CreatedUtc { get; set; }
    public string CreatedBy { get; set; } = string.Empty;
    public DateTime ModifiedUtc { get; set; }
    public string ModifiedBy { get; set; } = string.Empty;
}

// The canonical entity of Listing 2, regenerated after the groups are
// approved: CreatedUtc/ModifiedUtc now come from the base, the
// principals are added, the soft-delete columns Listing 2 already
// carried are now expressed through ISoftDeletable, and every other
// member — including HolderId and Version — is unchanged.
[Table("account")]
public class AccountEntity : AuditableEntity, ISoftDeletable, IVersioned
{
    public int Id { get; set; }
    public string AccountNumber { get; set; } = string.Empty;
    public int HolderId { get; set; }
    public decimal Balance { get; set; }
    public string Status { get; set; } = AccountStatus.Pending;
    public int Version { get; set; }

    public DateTime? DeletedUtc { get; set; }
    public string? DeletedBy { get; set; }
}
```

Listing 19. Schema-derived interfaces and an abstract base: recurring column groups become typed seams.

Interfaces are the composition mechanism; bases are a convenience. A table carrying the audit, soft-delete, and version groups cannot inherit three base classes in C#, but implements three interfaces without friction. The practical rule: emit interfaces always; emit an abstract base only for a dominant or strictly nested group; and let the generator maintain both, so the seams regenerate with the schema. In the inheritance variant business objects participate transparently — Account : AccountEntity is IAuditable through its base; in the composition variant the service stamps the unwrapped entity. Either way, centralized behavior applies to every write that passes through the base.

The payoff is that cross-cutting logic collapses into one governed location:

```
// One place knows how stamping, soft deletion, and read-model
// protection work. The raw provider is private; the user, clock, and
// audit writer are exposed to derived services on purpose.
public abstract class ServiceBase
{
    private readonly IOrmDataProvider _provider;
    protected ICurrentUser User { get; }
    protected IClock Clock { get; }
    protected IAuditWriter Audit { get; }

    protected ServiceBase(IOrmDataProvider provider, ICurrentUser user,
                          IClock clock, IAuditWriter audit)
    {
        _provider = provider; User = user; Clock = clock; Audit = audit;
    }

    protected Task<TKey> InsertAsync<T, TKey>(T entity)
        where T : class, new()
    {
        Guard(entity); Stamp(entity, isNew: true);
        return _provider.InsertAsync<T, TKey>(entity);
    }

    protected Task<T> UpdateAsync<T>(T entity) where T : class, new()
    {
        Guard(entity); Stamp(entity, isNew: false);
        return _provider.UpdateAsync(entity);
    }

    // Reads: a soft-deletable type cannot be queried unfiltered by
    // accident; the caller must say which it wants.
    protected IQueryable<T> Query<T>() where T : class, new()
    {
        if (typeof(ISoftDeletable).IsAssignableFrom(typeof(T)))
            throw new InvalidOperationException(
                $"{typeof(T).Name} is soft-deletable: use QueryActive " +
                "or QueryIncludingDeleted.");
        return _provider.Query<T>();
    }

    protected IQueryable<T> QueryActive<T>()
        where T : class, ISoftDeletable, new()
        => _provider.Query<T>().Where(e => e.DeletedUtc == null);

    protected IQueryable<T> QueryIncludingDeleted<T>()
        where T : class, ISoftDeletable, new()
        => _provider.Query<T>();

    private void Stamp(object entity, bool isNew)
    {
        if (entity is not IAuditable a) return;
        var now = Clock.UtcNow;
        if (isNew) { a.CreatedUtc = now; a.CreatedBy = User.Name; }
        a.ModifiedUtc = now; a.ModifiedBy = User.Name;
    }

    private static void Guard(object entity)
    {
```

```
        if (entity is IReadModel)
            throw new InvalidOperationException(
                "Read models are never written.");
    }
}
```

Listing 20. Centralized stamping, filtering, and read-model protection: the raw provider is private, so every write is stamped, every soft-deletable read is deliberate, and no projection can be written.

Consider the alternatives without these seams — the answer to "how else would audit fields be centralized?" is: unsatisfactorily. Per-service stamping repeats the same four assignments at every save and relies on review to catch omissions — precisely the human-discipline burden the pattern exists to remove (Section 3.3). Reflection-based stamping (locate properties named CreatedUtc by string) trades compile-time safety for convention magic and silently misses a renamed column. Database triggers centralize the write but sever it from application identity — the database sees a connection principal, not the application user, unless session context is explicitly smuggled per call — and they hide the logic from code review while duplicating it per engine. Full ORMs centralize stamping in SaveChanges interceptors, which works, at the price of re-adopting the event pipeline whose consequences Section 5 catalogs. The interface seam achieves the centralization with none of these costs: one implementation, compile-time checked, engine-neutral, visible in review, and unit-testable like any other class. Where database-side attribution is nonetheless required — row-level security predicates, trigger-written audit tables — the gap can be closed from the application side by priming the end-user identity into per-connection session context, so the database sees the application user rather than the connection principal; FunkyORM (3.8 and later) supplies this across its engines, and the same can be done by hand on any provider.

The open/closed payoff compounds over time. When a new table acquires the audit columns, the regenerated entity implements IAuditable, and every existing centralized behavior — stamping, audit-trail emission, deleted-row filtering — applies to it with no code modified. Interface segregation pays at the consumer end: an audit-trail writer that accepts IAuditable cannot couple itself to an entity's other forty properties, and a test can satisfy it with a five-line fake. The same technique extends beyond audit fields to any recurring group the schema exhibits — tenancy keys, residency markers, effective-date ranges — wherever a column set recurs, a typed seam can centralize its handling.

6.9 The Regulated Seam in Practice

The paper leans on regulated settings for its motivation, and a seam is only an opportunity until something is attached to it. Listing 21 shows a read in a healthcare

context doing four things the regulation actually asks for: a consent check before any protected data is materialized; a projection that cannot select a column it must never return; row-level security applied by the database from the application user's identity; and an audit record of the access at the same call site.

```
// Canonical entity (generated) holds everything the table holds,
// including a column that must never leave the service.
[Table("patient")]
public class PatientEntity : AuditableEntity
{
    public int Id { get; set; }
    public string Mrn { get; set; } = string.Empty;
    public string Ssn { get; set; } = string.Empty;    // never projected
    public string Metadata { get; set; } = string.Empty;
}

// Detail type: the minimum-necessary shape for a care-team view.
// Ssn is not a member, so no query through this type can select it.
[Table("patient")]
public sealed class CareTeamPatientView : IReadModel
{
    public int Id { get; set; }
    public string Mrn { get; set; } = string.Empty;

    [JsonPath("metadata", "$.primaryTeam")]
    public string PrimaryTeam { get; set; } = string.Empty;
}

// Attribute-road service over the ServiceBase of Listing 20.
public sealed class PatientAccessService : ServiceBase
{
    private readonly IConsentService _consent;

    public PatientAccessService(IOrmDataProvider provider,
        ICurrentUser user, IClock clock, IAuditWriter audit,
        IConsentService consent)
        : base(provider, user, clock, audit) => _consent = consent;

    public async Task<CareTeamPatientView?> GetForCareTeamAsync(
        int patientId, Purpose purpose, Guid correlationId)
    {
        // 1. Consent is checked before anything is materialized.
        await _consent.RequireAsync(User.Id, patientId, purpose);

        // 2. The database applies row-level security from the
        //    application user, primed into session context for this
        //    request (sp_set_session_context on SQL Server, set_config
        //    on PostgreSQL; the attribute road automates the priming).
        var view = Query<CareTeamPatientView>()
            .FirstOrDefault(p => p.Id == patientId);

        // 3. The access itself is an auditable event.
        await Audit.WriteAsync("Patient.Read", patientId, correlationId,
                               purpose: purpose.ToString());
        return view;
    }
}
```

Listing 21. The regulated seam doing regulated work: consent before materialization, a projection that cannot select the forbidden column, row-level security from the application user, and an audited access.

The second step deserves the emphasis. A column that must never leave the service is protected here not by a rule, a filter, or a review comment, but by absence: the view type has no member for it, so no query through that type can select it, and a search for the member name finds every place it is permitted. Row-level security closes the gap Section 6.8 named — the database seeing a connection principal rather than the user — by priming the user's identity into per-connection session context; the mechanism exists natively on SQL Server and PostgreSQL, and the attribute road automates the priming. None of this is unique to Slim Domain; every service layer could do it. What the pattern contributes is that the seam is the intended read path — the service base keeps the provider private and the architecture test keeps the library out of other namespaces — so a consent check or an audit write can be skipped only by writing a second read in the same service, which review and a fitness function can see; and that the minimum-necessary shape is a type, not a discipline. (A service can still query the canonical PatientEntity; the base's query path, not the type system, is where that is governed.)

---

7. Consequences and Required Discipline

This section states the pattern's consequences in one place: what it provides structurally, what it may enable, what it costs, and what discipline each cost demands. Section 8 then compares those demands with classic n-tier and with modern baselines.

7.1 What the Pattern Provides

- **Defect surfaces removed by construction.** No navigation properties in the persistence model and no change tracker means the N+1, lazy-load, and stale-tracking defect classes have no syntax to inhabit; in Appendix B's legacy comparison the corresponding constructs measure zero.
- **One explicit, auditable persistence seam.** Every load and save is a visible call site where auditing, consent, encryption, and the version predicate attach (Listings 5 and 21), and persistence participation concentrates into a handful of files.
- **A small reading surface.** Four concepts suffice to read the code (Section 3.1); the writing curriculum is Section 6.

- Low mapping ceremony. No DTO or mapper layer between layers that share a process; against the legacy baseline, schema-mapping attribute ceremony per line is roughly twelve times lower (Appendix B).
- Centralized rules. Behavior on business objects, policy at the service seam, and cross-cutting column groups on schema-derived interfaces (Section 6.8), so validation, stamping, and filtering live in a few governed places.
- Guardrails for mixed-skill teams. The dangerous paths are structurally inconvenient rather than merely discouraged, which reduces reliance on perfect human performance.
- Predictable performance. Stateless execution and explicit query shapes; nothing is loaded that was not asked for.
- Bounded interest on cross-cutting change. Because persistence, cross-cutting rules, and read shapes each have one home, the files touched by a schema change, an audit requirement, or a new screen are bounded by construction — the debt-prevention consequence Section 3.4 develops.

7.2 What It May Enable

Because all writes already pass through a seam, event sourcing and command-query separation are incremental additions rather than rearchitecture, with detail entities serving as read models. Minimal entities and per-service gates make service decomposition cheaper, and engine-portable providers let the same entities serve production, edge, and test databases. The small concept count makes the pattern teachable. Two further prospects are hypotheses rather than results and are labeled as such: that convention-driven data access is easier for coding agents to generate correctly than stateful ORM code, and that the concentrated audit surface shortens audit cycles in regulated settings (Listing 21); Section 11.2 proposes measuring the second.

7.3 Costs and the Discipline They Require

Table 3 lists the pattern's weaknesses and the compliance-oriented threats of regulated settings together, because the question for each is the same: what added discipline does it demand, and does the pattern help or hinder applying it? Two rows read "Impedes" — encapsulation in the inheritance variant and graph-navigating consumers — and they are the honest ones.

Concern	Why it matters	Added-discipline solution	Effect of Slim Domain on the solution
Inheritance / schema coupling	Business objects inherit the persisted shape, so schema changes surface directly in the domain layer.	Regenerate entities through tooling and treat schema changes as reviewed contract changes. Compile errors localize drift immediately. Section 6.5 examines this in depth.	Facilitates — drift is visible and cheap to correct at compile time rather than hidden in mapping configuration.
Encapsulation / invariant enforcement	Inherited properties carry public setters, so invariants are not enforced by the type system alone.	Route mutation through intention-revealing methods; validate within business objects; in regulated domains, default to the composition variant (Section 4.2), which removes the bypass vector instead of policing it.	Impedes in the inheritance variant — public setters are a bypass the type system does not close; neutral in the composition variant, which closes it structurally.
Advanced persistence primitives	Change tracking, optimistic concurrency, lazy loading, and second-level caching are not supplied by the micro-ORM.	Add version columns, explicit cache layers, and explicit queries only where measurement justifies them. Section 6.2 shows the transaction primitives that are supplied.	Facilitates — primitives are added deliberately, without inheriting a full ORM's hidden state and side effects.
Generator / scaffolding dependency	The pattern leans on entity generation from the schema; the generator becomes critical infrastructure.	Maintain the generator as first-class infrastructure; verify generated entities against a schema snapshot (e.g., an INFORMATION_SCHEMA comparison) in continuous integration, so drift fails the build rather than surprising a developer.	Tie — classic n-tier scaffolding (EF migrations, reverse engineering, T4) carries an equivalent dependency.
Graph-navigating consumers	GraphQL resolvers and OData $expand promise clients arbitrary traversal; flattened reads serve only the shapes that were written.	Compose resolvers from narrow reads; cap traversal depth; accept that the host style stops doing what it was designed to make easy (Section 2.5).	Impedes — this is the pattern's honest friction case.
Fat services / anemic-domain drift	Behavior can accrete in services over time, leaving business objects anemic.	Adopt a behavior-first placement rule (logic belongs on the business object unless it spans aggregates); enforce it in review and encode it as an architectural fitness function (Section 9.4).	Tie — service-heavy drift afflicts layered Entity Framework solutions equally.
Auditability / immutability / event sourcing	Regulators expect immutable, reconstructable histories of sensitive operations.	Emit domain events or audit records at the service seam; use append-only stores where mandated.	Facilitates — a single explicit persistence seam gives one place to attach and verify audit emission.

Concern	Why it matters	Added-discipline solution	Effect of Slim Domain on the solution
Encryption / tokenization / consent	Protected health information and cardholder data must be encrypted or tokenized, and consent verified before materialization.	Perform consent checks and cryptographic operations at the service boundary, before business objects are populated.	Facilitates — data lifetime is explicit; no proxies or tracked graphs hold decrypted state in memory.
Fine-grained authorization / ABAC	Row- and column-level access decisions must be applied consistently across every read path.	Centralize policy evaluation in services, or decorate queries with policy predicates.	Tie — global query filters in full ORMs offer partial help but introduce their own bypass risks.
Complex transactions / sagas	Multi-step operations across aggregates or services need atomicity or compensation.	Use explicit transaction scopes within services (Section 6.2); adopt outbox or saga patterns for cross-service flows.	Tie — both patterns require the same deliberate machinery.
Regulatory reporting expectations	Auditors expect documented data lineage and reproducible reporting flows.	Maintain explicit query catalogs and documented data-flow diagrams.	Tie — explicit SQL is straightforward to catalog, but the documentation burden is identical.
Auditor / team resistance	An unfamiliar solution shape can draw findings or slow review, independent of technical merit.	Provide architecture documentation; apply the rename-only restructuring and the optional repository layer (Section 9).	Classic n-tier is favored short-term on familiarity; the presentation options neutralize the difference.

Table 3. Costs, threats, and the discipline each requires. "Impedes" rows mark where the pattern works against the mitigation.

8. Comparative Analysis: Slim Domain vs. Strict N-Tier

8.1 Summary Comparison

Weakness / threat	Does strict n-tier automatically solve it?	Requires deliberate extra mechanisms in both?	Which pattern makes the mechanism easier to apply?
Inheritance / schema coupling	No	Yes	Slim (visible and cheap)
Encapsulation / invariant enforcement	No	Yes	Classic n-tier (inheritance variant); Tie (composition variant)

Weakness / threat	Does strict n-tier automatically solve it?	Requires deliberate extra mechanisms in both?	Which pattern makes the mechanism easier to apply?
Graph-navigating consumers	Yes, with a full ORM	No	Classic n-tier with a full ORM
Advanced persistence primitives	Only via heavy ORM (with side effects)	Yes	Slim (add only what you need)
Generator / scaffolding dependency	No	Yes	Tie
Fat services / anemic drift	No	Yes	Tie
Auditability / immutability / event sourcing	No	Yes	Slim (explicit services)
Encryption / tokenization / consent	No	Yes	Slim (explicit data lifetime)
Fine-grained authorization / ABAC	No	Yes	Tie
Complex transactions / sagas	No	Yes	Tie
Regulatory reporting expectations	No	Yes	Tie
Auditor / team resistance	Partially (familiarity)	Yes (documentation)	Classic n-tier (short-term)

Table 4. For each weakness or threat: whether strict n-tier solves it automatically, whether both patterns require deliberate mechanisms, and which pattern applies the mechanism more easily.

8.2 Assessment

Strict n-tier layering by itself is not a silver bullet for any of the significant weaknesses or threats discussed above. In almost every case, the architect or developer must deliberately apply additional mechanisms — the same ones required by Slim Domain.

The two patterns are much closer in required discipline than the surface debate suggests. The substantive differences are two:

- Slim Domain makes persistence explicit and stateless by design, which removes an entire class of hidden bugs (N+1 queries, change-tracking side effects) and makes audit and encryption points easier to insert and verify.
- Classic n-tier with Entity Framework supplies more “free” ORM features (change tracking, lazy loading, global filters) but also inherits their

risks and complexity — and most compliance mechanisms still must be added on top.

The paired-system measurements of Appendix B illustrate the mechanism without settling the comparison. In the classic-stack system, roughly 550 business-layer files define active-record Save() methods — persistence as ambient capability — while the Slim Domain system's persistence surface concentrates in seven files. That contrast measures one tenet, that nouns do not save themselves, against a legacy baseline; it does not compare the pattern with a modern service-gated EF Core or Dapper codebase, which would share the same absences. Section 8.3 takes up that comparison on mechanism rather than measurement. Meanwhile cyclomatic complexity at the business-and-data layer is statistically indistinguishable between the two systems, undercutting the suggestion that the explicitness is paid for in complexity.

Comparing Slim Domain plus the necessary discipline against classic strict n-tier plus the necessary discipline, the Slim pattern does not create meaningfully more work for the hard problems — compliance, invariants, encryption, audit. In several areas it reduces accidental complexity, making the deliberate mechanisms easier to apply correctly and to demonstrate to regulators. Neither pattern solves these problems on its own; both require thoughtful engineering on top of the layering.

8.3 Against Modern Baselines

Strict n-tier with designer-era Entity Framework is the stack many regulated codebases still run, and the stack Appendix B measured; it is not the stack a 2026 team is choosing against. Two modern baselines deserve the comparison directly, and against both the pattern's delta is smaller than the legacy contrast suggests.

A disciplined EF Core service layer — lazy loading off, SaveChanges confined to handlers, split queries and ExecuteUpdate where they help, interceptors as the cross-cutting seam. What remains of the pattern against this baseline: reads shaped to the use case by default rather than by vigilance (detail entities and views instead of Include graphs that are merely well managed); a canonical entity that stays minimal because richer shapes have somewhere else to live; business objects that acquire the entity shape without a mapping layer, with the DTO warrant test governing the exceptions; the version predicate and the audit write visible at the call site rather than inside a SaveChanges pipeline; and schema-derived interfaces instead of interceptors that match on type. The change tracker is the honest residual difference: it remains available as a

bypass — a tracked entity mutated anywhere is persisted by the next SaveChanges — so the structural guarantee becomes a convention held by fitness functions (Section 2.6).

A Dapper vertical slice — one handler per feature, explicit SQL, a request and a response type per slice. Most of the pattern is already present here: persistence lives in the handler, reads are explicit, nouns do not save themselves. The delta is the DTO warrant test (slices tend to mint a DTO per slice whether or not a boundary exists), the canonical/detail distinction (slices tend to accrete columns onto shared types), behavior on the business object rather than inline in the handler, and schema-derived interfaces that give cross-cutting rules a home a slice cannot forget. Such a team adopts the pattern by adopting its naming and its tests, not by changing libraries.

Stated against these baselines the pattern is an incremental discipline, not a rival architecture — which is what Section 2 claims it is. The large structural contrasts of Appendix B belong to the legacy comparison and should be read there.

---

9. Presenting Slim Domain in N-Tier Terms

A point that the debate around the pattern often misses: Slim Domain is not architecturally incompatible with n-tier. Its dependency flow (Figure 5) is already layered — presentation, application, domain, infrastructure contracts — and Section 9.2 shows that a conventional n-tier appearance is achievable through renaming alone. What the pattern actually changes is emphasis: it promotes different naming and organization practices in order to encourage different developer thought patterns, chiefly that persistence is a deliberate, gated act performed by services rather than an ambient capability available everywhere. This section therefore describes presentation options, not a conversion. Teams and auditors who prefer the conventional appearance can have it — immediately, reversibly, and without sacrificing the pattern's benefits. Figure 6 summarizes the path; the Aegis Platform again serves as the reference solution.

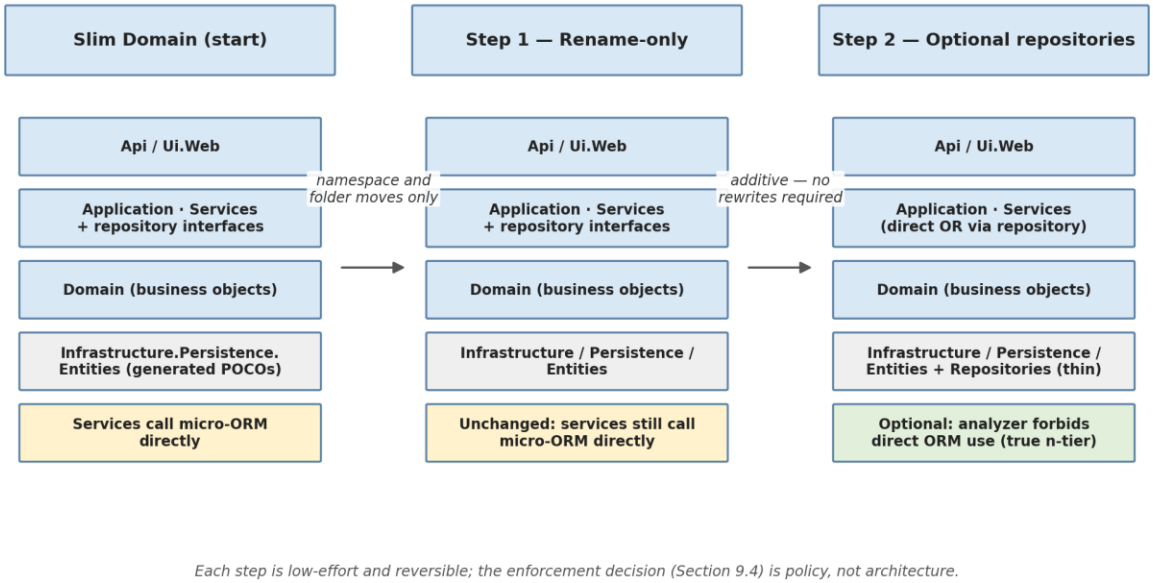


Figure 6. The presentation path: rename-only restructuring, an optional thin repository layer, and optional build-time enforcement.

9.1 Current Aegis Structure (Slim Domain Naming)

```
Aegis/
+-- Aegis.Api/                          # ASP.NET Core Web API
+-- Aegis.Ui.Web/                       # Blazor Server + MudBlazor
+-- Aegis.Application/                  # Orchestration layer
|   +-- Services/                       # Business rules, aggregation
|   +-- Repositories/                   # Interfaces (what services depend on)
+-- Aegis.Domain/                       # Business objects with behavior
+-- Aegis.Infrastructure.Persistence.Entities/   # Generated slim POCOs
+-- Aegis.Shared.Contracts/             # DTOs
+-- Aegis.ServiceDefaults/              # Aspire defaults
+-- Aegis.AppHost/                      # Aspire orchestration
+-- (standalone tooling and documentation projects)
```

Key characteristics: Aegis.Domain.Account acquires the shape of Aegis.Infrastructure.Persistence.Entities.AccountEntity — by composition in the ledger, by inheritance in CRUD modules; services call the data-access library directly (Dapper or FunkyORM) — low ceremony, explicit, stateless; and all persistence is service-gated.

9.2 Step 1: Rename-Only Restructuring (High Perception Gain, Zero Code Change)

The first step makes the structure communicate classic n-tier flow (Api → Application → Domain → Infrastructure) without touching any logic: Aegis.Domain and Aegis.Infrastructure.Persistence.Entities move into clearer top-level layers, and namespaces follow. No code changes are required — only namespace and folder moves.

Existing services continue to call the micro-ORM directly. That this step suffices to produce a conventional appearance is itself evidence for this section's framing: the layering is shared. What the pattern adds survives any naming — no in-process mapping without a boundary warrant, no ambient persistence, reads flattened to the use case, cross-cutting rules on schema-derived interfaces — and those rules, not the project names, are what an architecture board is being asked to adopt.

```
Aegis/
+-- Aegis.Api/
+-- Aegis.Ui.Web/
+-- Aegis.Application/
|   +-- Services/
|   +-- Repositories/               # Interfaces only
+-- Aegis.Domain/
+-- Aegis.Infrastructure/
|   +-- Persistence/
|       +-- Entities/               # Generated slim POCOs
+-- Aegis.Shared.Contracts/
+-- Aegis.ServiceDefaults/
+-- Aegis.AppHost/
+-- (standalone projects)
```

9.3 Step 2: Optional Thin Repository Layer (Full Classic Appearance)

The second step adds a thin implementation layer under Infrastructure for use when auditors or architects require "repositories in the data layer," while keeping the fast path available.

```
Aegis/
+-- Aegis.Api/
+-- Aegis.Ui.Web/
+-- Aegis.Application/
|   +-- Services/        # Call micro-ORM directly OR inject repository
|   +-- Repositories/    # Interfaces (unchanged)
+-- Aegis.Domain/
+-- Aegis.Infrastructure/
|   +-- Persistence/
|       +-- Entities/
|       +-- Repositories/             # NEW: thin optional implementations
|           +-- AccountRepository.cs  # Delegates to the ORM (terse)
|           +-- TransactionRepository.cs
|           +-- RepositoryBase<T>
+-- Aegis.Shared.Contracts/
+-- (standalone projects)
```

Default path (bypass mode). Aegis.Application.Services.AccountService continues to call the micro-ORM directly. Zero additional ceremony; all existing code works unchanged.

When required (repository mode). The service injects IAccountRepository (from Aegis.Application.Repositories); the implementation in Aegis.Infrastructure.Persistence.Repositories is a one- or two-line delegate to the micro-ORM.

Keeping repositories stateless. A repository in this pattern is a naming and testability convenience — a thin, stateless delegate. It must not become a reimplementation of the machinery the pattern deliberately omitted. In too many real-world codebases, repositories accrete until they resemble a hand-rolled DbContext; each of the following is a sign that this drift has begun:

- Entity caching or an identity map ("return the same instance for the same key")
- Pending-change buffers with a save-everything method — a reinvented SaveChanges
- Connection or transaction state held in fields beyond an explicitly scoped transaction
- Per-entity query zoos: dozens of FindByX methods accreting where services should own their queries

If a repository needs any of these, the need is real but the location is wrong: caching belongs in an explicit cache layer, units of work belong in service-scoped transactions (Section 6.2), and query logic belongs in services. The repository stays a delegate, or it stops paying for its existence.

9.4 Enforcement: Making the Policy Verifiable

Where both paths coexist, the choice between them must be a documented policy decision, not an accident of individual style — otherwise the dual path invites exactly the inconsistency finding it was meant to prevent. Two complementary mechanisms make the policy verifiable:

- Build-time prohibition. A Roslyn analyzer or architecture test (for example, a NetArchTest rule) fails the build if any type outside the approved namespaces references the micro-ORM. Enabling this rule yields fully conventional n-tier enforcement — as a one-line policy change, not an architectural rewrite.
- Architectural fitness functions. The same hook cheaply encodes drift detectors beyond ORM-reference prohibition: flag service methods whose non-orchestration logic exceeds a size threshold (fat-service drift); flag assignments to

business-object setters outside constructors and business methods (invariant bypass); and fail the build when generated entities diverge from a schema snapshot (generator drift). Each rule is a few lines of NetArchTest or analyzer code, and each converts a review-time convention from Section 7.3 into a build-time guarantee.

- Documented dual-path policy. Prefer a single path per solution: a pilot picks bypass mode or repository mode and writes the choice down, because the two coexist safely only under the fitness functions below. If bypass mode nonetheless remains permitted alongside repositories, the architecture documentation states which operation classes use which path and why, so that reviewers and auditors see deliberate governance rather than drift.

The build-time rule is a few lines of an architecture test, shown here with NetArchTest. Its approved-namespace list is the policy switch of Section 9.3: in bypass mode the services namespace is approved, as the flagship requires; in repository mode it is removed and the test fails on the first direct call. A read-model write guard is a comparable analyzer rule, described in Section 6.3:

```
// Architecture test: the persistence gate as a failing build. The
// approved list is the policy switch of Section 9.3 — bypass mode
// approves the services namespace (as the flagship requires);
// repository mode deletes that line and the first direct call fails.
private static readonly string[] ApprovedPersistenceNamespaces =
{
    "Aegis.Infrastructure.Persistence",
    "Aegis.Application.Services",          // bypass mode only
};

[Fact]
public void Only_approved_namespaces_reference_a_data_access_library()
{
    var result = Types.InAssembly(typeof(AccountService).Assembly)
        .That().DoNotResideInNamespaceMatching(string.Join("|",
            ApprovedPersistenceNamespaces.Select(Regex.Escape)))
        .ShouldNot().HaveDependencyOnAny("Funcular.Data.Orm", "Dapper",
                                         "Microsoft.EntityFrameworkCore")
        .GetResult();

    Assert.True(result.IsSuccessful,
        string.Join(", ", result.FailingTypeNames ?? Array.Empty<string>()));
}
```

Listing 22. An architecture test: the persistence gate as a failing build, with the bypass/repository policy as one list.

Because the repository layer and the enforcement hook are both additive, every step in this section is reversible. Switching a solution from bypass mode to repository mode is mechanical but not free — every direct call moves behind an interface — so the choice

is best made once, early, per solution. The enforcement decision itself is policy, and policies can be chosen per organization.

---

10. Related Work

Layered enterprise patterns. Slim Domain's service layer corresponds closely to Fowler's Service Layer pattern, and its slim entities resemble the record structures consumed by a Data Mapper [2]. It differs from Transaction Script in that behavior is retained on a dedicated domain layer rather than dissolved into procedures, and from Table Module in that business objects are instance-oriented rather than table-oriented.

The anemic-domain-model critique. Fowler's criticism of anemic domain models [3] targets architectures whose "domain" objects are bags of getters and setters with logic exiled to procedural services. Slim Domain is deliberately not that: the slim entities are persistence contracts, not the domain; behavior lives on the enriched business objects. The pattern does, however, share the risk that behavior drifts into services over time — Table 3 addresses this directly.

Domain-Driven Design. Slim Domain is compatible with DDD's strategic practices — ubiquitous language, bounded contexts — while relaxing its tactical prescriptions [4]. Aggregates, repositories, and factories become optional tools applied where they pay for themselves, rather than mandatory structure.

Clean and Onion Architecture. Clean Architecture [6] and Onion Architecture [5] place dependency inversion at the center: domain code must not depend on infrastructure abstractions. Slim Domain makes a deliberate, narrow exception — business objects depend on the entity shape — trading a measure of dependency purity for the elimination of an entire mapping layer. Section 6.5 treats the cost of that trade honestly; Section 9 shows that teams which later want the inverted shape can adopt it incrementally.

Vertical Slice Architecture. Bogard's Vertical Slice approach [7] organizes code by feature rather than by layer and is largely orthogonal to Slim Domain: slices can be cut across a Slim Domain solution, with each slice's handler playing the service role.

CQRS and event sourcing. Because every persistence operation already flows through an explicit service seam, introducing command-query separation or an event

store [8] is an incremental change rather than a rearchitecture — one of the pattern's design goals. The detail entities of Section 6.3 serve naturally as read models.

The current .NET decade. The ecosystem a 2026 reader is choosing within matters as much as the classics. Entity Framework Core's present feature set — keyless entity types mapped to views, no-tracking queries, split queries, ExecuteUpdate, and interceptors [19] — is what makes the view road of Section 2.6 practicable and is the baseline Section 8.3 compares against. Vernon's treatment of the aggregate as a consistency boundary [20] is the position Listing 13 answers with a transaction and per-row predicates. Mediator pipeline behaviors [21] are the ecosystem's prevailing cross-cutting seam; the pattern places equivalent rules on schema-derived interfaces and a service base instead, so that they attach to data shapes rather than to request types. The widely copied Clean Architecture solution template [22] is the shape many teams will be asked to reconcile the pattern with, and Section 2.4 does so. Transactional messaging frameworks such as Wolverine [23] supply the outbox and saga machinery Table 3 defers to. Finally, "functional core, imperative shell" [24] is the closest sibling of the nouns-and-verbs tenet: behavior on values, effects at the edge.

Technical debt. Cunningham's original metaphor [25] and Fowler's quadrant [26] supply the vocabulary of Section 3.4; Kruchten, Nord, and Ozkaya [27] give it theoretical footing. Ernst et al. [28] report from practitioners that architectural decisions, not code-level sloppiness, are the dominant source of debt — the finding on which the pattern's debt claim rests. Ford, Parsons, and Kua [29] frame fitness functions as the instrument against architectural erosion, which is what Section 9.4 adopts them for; Tornhill's hotspot analysis [30] — change frequency weighted by complexity — is the measurement the roadmap of Section 11.2 borrows for accrual rate.

11. Limitations and Validation Roadmap

11.1 Limitations

Intellectual honesty requires stating the limits of the evidence offered and of the pattern itself.

- Structure is not outcome. Appendix B measures structural surface — complexity, ceremony, dispersion, the presence or absence of bug-prone constructs — across one paired set of production systems. It does not measure defect rates,

onboarding time, or audit outcomes, and a single pair cannot support generalization. Claims of faster onboarding and lower bug-injection rates therefore remain field reports pending the measurements proposed in Section 11.2.

- The paired comparison is observational. The two systems in Appendix B differ in domain, age, scale, and platform era; differences cannot be attributed to architecture alone. Its baseline is a legacy designer-era Entity Framework system with active-record persistence, and the absences it records — eager loads, change tracking, ambient saves — would also be absent in any service-gated EF Core or Dapper codebase. The comparison therefore supports the pattern's tenets within one lineage and cannot rank the pattern against its modern alternatives, which Section 8.3 compares on mechanism. It quantifies what each architecture obliges developers to write — no more.
- Single-lineage experience. The deployments informing this paper — including both systems measured in Appendix B — were led by, or influenced by, the originating practitioners. Independent adoption data does not yet exist.
- Schema churn: parity, paid early. Inheriting the generated shape couples the domain layer to the schema, and every persistence strategy carries that coupling somewhere — migrations and entity configuration, mapper layers, SQL strings, schema-on-read versioning. No pattern or library the authors know of avoids churn on a volatile schema; Slim Domain pays it at compile time in the consuming layer (Section 6.5). The one genuine delta is that a domain project recompiles for columns it does not use, which teams with volatile schemas or several consuming domains will notice more than the reference scenarios do.
- Debt prevention is claimed by construction, not from accrual data. Section 3.4 maps tenets to debt channels, and Appendix B measures the channels' structural proxies in one paired snapshot. Whether a Slim Domain codebase accrues debt more slowly over its life than a comparable modern codebase is unmeasured; Section 11.2 proposes the accrual metrics.
- Encapsulation depends on convention. In the inheritance variant, inherited public setters mean invariants are protected by convention and review rather than by the type system; the composition variant closes this gap at the price of additional ceremony.

- Snapshot validity of the feature matrix. Table 2 was compiled from project documentation in June 2026, with the FunkyORM column refreshed against release 3.12.1 in October 2026; the other columns were not re-verified at that time. Micro-ORM feature sets evolve quickly; readers should verify against current releases.
- Author affiliation. FunkyORM, used for the attribute-road examples in Sections 4.4 and 6, is maintained by the author's organization (disclosed in Section 1.4). The techniques shown have equivalents in other micro-ORMs, as noted throughout, but the choice of demonstration vehicle is not neutral.
- No public generator accompanies the paper. Section 4.6 specifies the generator contract the pattern relies on, and Section 6.8's interfaces were derived by hand in the deployments behind this paper. A team adopting from the paper alone can meet the contract with a script or a template, but the cross-cutting technique has not yet been exercised with a tool that implements the contract end to end.
- Ecosystem familiarity. Auditors, new hires, and tooling are calibrated to conventional n-tier and Entity Framework. Section 9 mitigates but does not eliminate this friction.

11.2 A Validation Roadmap

The structural measurements of Appendix B are a first installment, not a destination. The claims that remain unmeasured are measurable with modest instrumentation, and the pattern's adopters are better placed than its authors to measure them independently. The roadmap, in increasing order of effort:

- Onboarding time. Days from first commit to first unassisted production-merged feature, recorded per new hire and compared across teams or before/after adoption. Cheap to collect from version-control history alone.
- Defect-class incidence. Tag production incidents and code-review findings by class (N+1/lazy-load, stale-state/tracking, mapping drift, invariant bypass). The pattern predicts near-elimination of the first two classes specifically — a falsifiable claim.
- Audit-finding rates. For regulated deployments: findings per audit cycle attributable to data-access controls, before and after adoption, normalized by audit scope.

- Structural replication. The Appendix B metric pipeline is deliberately simple (a complexity analyzer plus pattern scans); replicating it on other paired systems would test whether the structural profile generalizes beyond this lineage.
- Debt accrual rate. Across releases: files touched per cross-cutting persistence change (a new audit column, a tokenization rule); growth of mapping attributes and [NotMapped] members per thousand lines per year; and hotspot overlap — change frequency weighted by complexity [30] — between persistence code and the rest of the system. These convert Section 3.4's structural claim into a longitudinal one.
- Controlled pilots and case studies. Anonymized case studies or paired-team pilots remain the standard the pattern should ultimately meet. The authors invite practitioners with comparable systems to contribute either; human peer review of this paper is likewise invited.

---

12. Conclusion

Slim Domain is a pragmatic pattern with a long production track record in environments ranging from internal tools to high-security regulated platforms. It was designed around the constraints architects actually face: mixed-skill teams we do not fully control, the need for fast onboarding, and the requirement for explicit, auditable persistence in regulated domains.

The pattern does not eliminate the need for deliberate engineering discipline; no pattern does. What it provides is a cleaner foundation on which that discipline is easier to apply, easier to audit, and harder to violate accidentally — and one that makes it significantly easier to centralize rules (business logic, validation, auditing, consent enforcement, encryption handling) in a few well-governed locations (Sections 4.3 and 6.8) rather than scattering them across disconnected entities, DTOs, and mapping layers.

The practical objections most often raised against the pattern — object-graph loading, transactions and cross-entity consistency, multi-table shapes, mapping exceptions, testing, optimistic concurrency, regulated reads — have working answers, shown in code in Section 6, and the micro-ORM ecosystem's trajectory (Table 2) is steadily narrowing the feature distance to full ORMs without importing their hidden state. The

consequences of Section 7 and the comparison of Section 8 show that the mitigations Slim Domain requires are comparable to — and in several cases easier than — those required by classic strict n-tier with Entity Framework; the paired-system measurements of Appendix B describe the structural side of that claim against a legacy baseline. The pattern's genuine costs are stated in Section 11 rather than minimized, and Section 11.2 lays out how the remaining claims should be tested.

Nor does the pattern force a presentation choice: as Section 9 shows, Slim Domain is not incompatible with conventional n-tier — it promotes different naming practices to encourage different developer thought patterns, and a team that wants the conventional appearance, thin repositories, or full build-time enforcement can adopt each, reversibly, without rework.

Read as technical debt, the pattern's claim is narrow and checkable: the common forms of inadvertent debt — over-fetching, parallel shapes, ambient persistence, scattered cross-cutting rules — are structurally unavailable, and the interest on cross-cutting change is bounded by construction. Schema churn, the cost most often raised against the pattern, is paid under every persistence strategy; the pattern's choice is only to pay it earliest. The accrual-rate measurements of Section 11.2 are how the claim would be confirmed or refuted.

Against the stated costs stand the benefits: the structural absence of N+1 and state-related defect surfaces, an order-of-magnitude reduction in mapping ceremony, persistence concentrated into an auditable handful of files (each measured in Appendix B against a legacy baseline), explicit audit seams, a straightforward path to event sourcing and CQRS, and strong guardrails for mixed-experience teams. For teams that value explicit control, low ceremony, strong guardrails, and the ability to demonstrate correctness to regulators, Slim Domain is a serious candidate for a bounded trial — the evidence in this paper supports that much and no more — and Section 11.2 describes the measurements that would say whether it should become a default.

References

[1] F. P. Brooks, Jr., "No Silver Bullet: Essence and Accidents of Software Engineering," IEEE Computer, vol. 20, no. 4, pp. 10–19, 1987.

[2] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA: Addison-Wesley, 2002.

[3] M. Fowler, "AnemicDomainModel," martinfowler.com, Nov. 2003. Available: https://martinfowler.com/bliki/AnemicDomainModel.html

[4] E. Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston, MA: Addison-Wesley, 2003.

[5] J. Palermo, "The Onion Architecture," jeffreypalermo.com, Jul. 2008.

[6] R. C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design. Boston, MA: Prentice Hall, 2017.

[7] J. Bogard, "Vertical Slice Architecture," jimmybogard.com, Apr. 2018.

[8] G. Young, "CQRS Documents," 2010.

[9] Stack Exchange, Inc., "Dapper — a simple object mapper for .NET," GitHub repository. Available: https://github.com/DapperLib/Dapper

[10] U.S. Department of Health and Human Services, HIPAA Privacy and Security Rules, 45 C.F.R. Parts 160 and 164, including the minimum-necessary standard at §164.502(b).

[11] PCI Security Standards Council, Payment Card Industry Data Security Standard (PCI DSS), v4.0.1, Jun. 2024.

[12] Funcular Labs, "FunkyORM — a lambda-powered .NET micro-ORM for MSSQL, PostgreSQL, MySQL & SQLite," GitHub repository. Available: https://github.com/FuncularLabs/Funcular.FunkyOrm

[13] M. Pendon, "RepoDB — a hybrid ORM library for .NET," GitHub repository. Available: https://github.com/mikependon/RepoDb

[14] Collaborating Platypus, "PetaPoco — a tiny ORM-ish thing for .NET," GitHub repository. Available: https://github.com/CollaboratingPlatypus/PetaPoco

[15] SqlKata contributors, "SqlKata Query Builder," project documentation. Available: https://sqlkata.com

[16] Linq To DB team, "LinqToDB — fastest LINQ database access," GitHub repository. Available: https://github.com/linq2db/linq2db

[17] ServiceStack, "OrmLite — fast, simple, typed ORM for .NET," project documentation. Available: https://docs.servicestack.net/ormlite/

[18] R. C. Martin, Agile Software Development: Principles, Patterns, and Practices. Upper Saddle River, NJ: Prentice Hall, 2003.

[19] Microsoft, "Entity Framework Core documentation: keyless entity types, tracking vs. no-tracking queries, split queries, ExecuteUpdate, and interceptors." Available: https://learn.microsoft.com/ef/core (accessed Sep. 2026).

[20] V. Vernon, Implementing Domain-Driven Design. Upper Saddle River, NJ: Addison-Wesley, 2013.

[21] J. Bogard, "MediatR — pipeline behaviors," GitHub repository documentation. Available: https://github.com/jbogard/MediatR

[22] J. Taylor, "Clean Architecture Solution Template for ASP.NET Core," GitHub repository. Available: https://github.com/jasontaylordev/CleanArchitecture

[23] JasperFx Software, "Wolverine — next-generation command and message bus for .NET," project documentation. Available: https://wolverinefx.net

[24] G. Bernhardt, "Boundaries" (the "functional core, imperative shell" formulation), talk, 2012. Available: https://www.destroyallsoftware.com/talks/boundaries

[25] W. Cunningham, "The WyCash Portfolio Management System," OOPSLA '92 experience report, ACM SIGPLAN OOPS Messenger, vol. 4, no. 2, pp. 29–30, 1992.

[26] M. Fowler, "TechnicalDebtQuadrant," martinfowler.com, Oct. 2009. Available: https://martinfowler.com/bliki/TechnicalDebtQuadrant.html

[27] P. Kruchten, R. L. Nord, and I. Ozkaya, "Technical Debt: From Metaphor to Theory and Practice," IEEE Software, vol. 29, no. 6, pp. 18–21, 2012.

[28] N. A. Ernst, S. Bellomo, I. Ozkaya, R. L. Nord, and I. Gorton, "Measure It? Manage It? Ignore It? Software Practitioners and Technical Debt," in Proc. ESEC/FSE 2015, pp. 50–60.

[29] N. Ford, R. Parsons, and P. Kua, Building Evolutionary Architectures: Support Constant Change. Sebastopol, CA: O'Reilly, 2017.

[30] A. Tornhill, Software Design X-Rays: Fix Technical Debt with Behavioral Code Analysis. Raleigh, NC: Pragmatic Bookshelf, 2018.

Appendix A: Review Methodology and AI Assistance Disclosure

A.1 Adversarial Review Process

Drafts of this paper and of the pattern's argumentation were subjected to multiple rounds of critique using large language models from more than one vendor — xAI's Grok (4.2 and 4.3 beta on the early drafts, Grok 4.7 for the later rounds) and Anthropic's Claude (Fable 5.1 and Opus 5.5) — prompted to argue against the pattern from four perspectives in turn: a software architect committed to Clean Architecture, a compliance auditor reviewing a regulated platform, a junior developer encountering the codebase for the first time, and a regulator assessing data-protection controls. Each round produced objections; the author accepted, rebutted, or scoped each one, and the surviving material informed Sections 6 through 8 of this paper. Model versions were updated as more capable models became available; the versions named are those in use at the time of writing. The structural measurements in Appendix B were likewise collected and aggregated with AI tooling assistance, under the author's direction, with identities of the measured systems withheld.

This process proved useful for surfacing objections systematically, and the breadth of critique exceeded what a single human reviewer typically produces in comparable time. It is, however, a drafting aid — not evidence of the pattern's validity, and not a substitute for human peer review or for the empirical validation acknowledged as absent in Section 11.

A.2 Authorship and AI Assistance

Consistent with emerging publication norms — under which AI systems do not meet authorship criteria because they cannot take responsibility for the work — this paper lists only its human author. AI assistance in drafting, critique, data aggregation, and revision is acknowledged here instead. All technical claims, recommendations, and conclusions are the author's responsibility.

Appendix B: Structural Metrics from Paired Production Systems

B.1 Systems and Method

Two production .NET systems were measured. System A is a mature line-of-business platform in a regulated domain, built on Entity Framework (designer-model era) with a rich domain model and substantial legacy active-record persistence, comprising a desktop client, a web client, and shared business and data layers. System B is a recent system in a different regulated domain, built on the Slim Domain pattern with a micro-ORM, comprising an API, a web UI, application services, domain objects, generated slim entities, and several processing applications. System identities are withheld; only aggregates are reported.

Complexity metrics were produced by the lizard analyzer (version 1.23, C# parser); pattern counts by regular-expression scans over non-generated source, excluding designer and code-generation outputs, build artifacts, and vendored packages. Test code is excluded from all production slices and characterized separately in B.5. The "biz+data" slice isolates each system's business-logic and data-access layers, excluding UI, host, and tooling projects — the fairest cross-system comparison, since System A carries a large desktop UI with no analogue in System B. Counts of generated entity layers are included on both sides, since each is code its pattern obliges the team to own.

Because the systems differ by roughly seventeenfold in size, raw counts are never compared directly. All cross-system comparisons below are made on normalized rates (per 100,000 nonblank lines, "/100k NBL") or on proportions; raw counts appear only as context. The measurement pipeline is small enough to replicate in an afternoon, and Section 11.2 invites exactly that.

B.2 The Architectural Signal

The central measurement is what each architecture obliges developers to write, maintain, and review per line of business and data code. Six constructs were counted, each chosen because it is either pure ceremony (mapping attributes) or a documented defect surface (navigation collections, change tracking, eager loading, ambient persistence methods). The rates are size-independent; the ratio column is the comparison.

Metric (biz+data layers, production code)	A — per 100k NBL	B — per 100k NBL	Ratio A:B	Raw counts (A vs. B)
Schema-mapping attributes ([Column], [Key], [Table], ...)	3,750	298	12.6×	10,843 vs. 27
[NotMapped] attributes	627	132	4.7×	1,814 vs. 12
Virtual collection navigation properties	270	11	24.5×	780 vs. 1
Active-record Save() definitions	202	0	—	585 vs. 0
Change-tracking operations (EntityState / Attach / AsNoTracking)	43	0	—	125 vs. 0
Eager-load (.Include) call sites	20	0	—	57 vs. 0

Table 5. The architectural signal: persistence and mapping ceremony per 100,000 nonblank lines of business and data code.

Two readings of this table matter, and both are readings of technical debt (Section 3.4): the ceremony rows measure accreted principal — shapes and attributes maintained on every change — and the zero rows measure channels through which inadvertent debt cannot accrue. First, the ceremony rows (mapping attributes, [NotMapped]) quantify a cost that is usually argued anecdotally: per line of comparable code, the classic stack carries roughly an order of magnitude more mapping ceremony — every attribute an opportunity for drift between code and schema, and every one subject to review. Second, the zero rows are not low counts; they are structural absences. System B contains no

change-tracking operations, no eager-load sites, and no active-record persistence methods because the constructs do not exist in its architecture. The defect classes associated with those constructs — N+1 query storms, stale-tracking surprises, save-side effects — have no syntax to inhabit. That is the mechanism of Section 3.2 in measured form, with the limit B.6 states: the absence distinguishes this codebase from its legacy predecessor, as it would distinguish any codebase that left Entity Framework, and does not by itself distinguish the pattern from a modern service-gated baseline. What the measurement supports is that nothing took the absent constructs' place.

B.3 Persistence Dispersion

The single strongest statistic in this study is where persistence lives. Table 6 counts the files that participate in persistence — referencing an ORM, using raw ADO.NET, or defining active-record save methods.

Metric (biz+data layers, production code)	System A	System B
Files referencing the ORM directly	6	6
Files using raw ADO.NET (SqlCommand etc.)	8	1
Files defining active-record Save() methods	≈550	0
Files participating in persistence (union)	≈560	7
Production files in slice	4,538	188
Share of files participating in persistence	≈13%	≈4%

Table 6. Persistence dispersion. Both systems gate direct ORM access through a comparable handful of files; the difference is the periphery — in System A, persistence is additionally an ambient capability of the business layer itself.

The first row deserves emphasis precisely because it shows parity: both systems concentrate direct ORM references into about half a dozen files. Gating the ORM is not what distinguishes the architectures — disciplined teams achieve that under any pattern. What distinguishes them is everything around that core. In System A, roughly 550 additional business-layer files define their own Save() methods: every aggregate is also a persistence actor, and the true persistence surface is some 560 files — about one production file in eight. In System B the union is seven files, all but one inside a single data-access project.

The operational consequences are concrete, and in debt terms they are the change-amplification factor — the interest rate on every cross-cutting persistence change (Section 3.4). For an auditor verifying that encryption, consent, or audit-trail rules hold

on every write path, the review surface is seven files versus roughly 560. For a cross-cutting persistence change — a new audit column, a tokenization rule, a correlation-identifier requirement — the modification surface differs by the same two orders of magnitude, and in System A each touched site is an opportunity for inconsistent application. For defect investigation, a persistence bug in System B originates in one project by construction; in System A it may originate anywhere in the business layer. And for enforcement (Section 9.4), the build rule "no persistence outside approved namespaces" is already true in System B and trivially verifiable, whereas the equivalent rule in System A would begin with several hundred violations to remediate.

The share row understates the contrast. System B's ≈4% is seven files concentrated in one place; System A's ≈13% is hundreds of files distributed across the business layer. Concentration, not just proportion, is what converts persistence from an ambient capability into a deliberate, gated act — the distinction Section 9 argues is the pattern's essential difference from its classic counterpart.

B.4 Complexity: A Control, Not a Scoreboard

Complexity metrics are reported to test an objection, not to rank the systems. The suspicion would be that Slim Domain's explicitness — hand-written queries, explicit composition, explicit transactions — pushes complexity into the service layer that a full ORM would otherwise absorb. At the comparable biz+data layer, the measurements do not support that suspicion: mean cyclomatic complexity, tail complexity, function length, and oversized-file rates are statistically similar.

Metric	A — full	B — full	A — biz+data	B — biz+data
Production files	6,098	349	4,538	188
Production nonblank lines (NBL)	494,335	28,636	289,182	9,069
Functions analyzed	25,342	1,045	12,298	385
Mean cyclomatic complexity (CCN)	2.2	3.5	2.0	2.2
95th-percentile CCN	6	12	6	6
% functions with CCN > 10	1.8%	6.4%	1.6%	3.1%
Mean function length (NLOC)	10.2	15.4	9.3	8.4
Files > 1,000 lines, per 100k NBL	8.7	10.5	9.3	11.0

Table 7. Scale and complexity context. Whole-solution figures are not comparable across systems (System B's are raised by genuinely algorithmic signal-processing code); the biz+data columns are the meaningful pair, and they read as parity.

The defensible conclusion is narrow and useful: in this pair, adopting the explicit style cost nothing measurable in code complexity at the layer where the two architectures do the same job. Domain, not architecture, dominates complexity — which is also why this table is a control rather than a headline.

B.5 Test Corpora

The systems' test corpora are not comparable and are excluded from every table above. System A's test volume (≈432,000 nonblank lines) is dominated by a recorded UI-automation suite — largely generated code with under two hundred attributed test methods. System B carries 679 hand-written unit and integration test methods in roughly 11,000 nonblank lines. No conclusion about testing rigor in either direction should be drawn from volume.

B.6 Threats to Validity

- The comparison is observational. The systems differ in domain, age, scale (A is roughly 17x B by production lines), and platform era; differences cannot be attributed to architecture alone.
- The measurements quantify structural surface — what each architecture obliges developers to write and maintain — not defect rates or other outcomes (Section 11.1).
- Pattern counts are regular-expression approximations; the active-record figure may include a small number of non-persistence Save methods. Spot checks indicate the orders of magnitude are robust.
- Both systems share practitioner lineage with this paper (Section 11.1); replication on independent systems is invited (Section 11.2).
- Complexity comparisons across systems are confounded by domain. The defensible reading is parity at the biz+data layer, not superiority of either system.
- The baseline is a legacy stack. The zero rows of Table 5 are produced by any codebase that does not use Entity Framework and does not place Save() on entities; they distinguish this lineage from its past, not Slim Domain from a modern service-gated EF Core or Dapper codebase.